



UNIVERSITY OF HERTFORDSHIRE

SCHOOL OF PHYSICS, ENGINEERING, AND

COMPUTER SCIENCE

MSc in CYBERSECURITY

7COM1039-0509-2024 - Cybersecurity Masters Project

September 2025

**HYBRID FORENSIC FRAMEWORK FOR
DATA BREACH ANALYSIS**

Name : BHARADWAJ AGRAHARAM

Student ID : 22099969

Supervisor : MOHAMMAD YAQOOB

ACKNOWLEDGEMENT

I wish to express my sincere gratitude to my supervisor “Dr. Mohammad Yaqoob” for their informative guidance, constructive criticism, and support throughout the duration of this project. Their professionalism was paramount in ensuring to guide the scope and consistency of my research. I also thank my lecturers, colleagues, and friends who gave me scholarly aid, logical deliberations, and beneficial resources that added legitimacy to this work.

Personally, I would also like to express gratitude to my family members and friends for their understanding, patience, and immense belief in me during this challenging but enriching experience. Their concern, motivation, and unwavering support provided the stimulus to finish this dissertation.

MSc FINAL PROJECT DECLARATION

I certify that this dissertation was done as part of the MSc Cybersecurity masters project requirement of University of Hertfordshire. There were not any human participants or sensitive personal data utilized within this project, and there was no ethical approval required, therefore. Any software used, data sets, and tools in the course while conducting this research were utilized for academic and non-business purposes only. I hereby affirm the work that has been submitted as my own, conducted in accordance with the university rules, and that all sources that were used have been accurately referenced.

I hereby give permission for the report to be made available on the University of Hertfordshire website provided the source is acknowledged.

I state that I have critically proof-read, and quality checked my report.

TABLE OF CONTENTS

ABSTRACT.....	1
1. INTRODUCTION	2
1.1 OVERVIEW	2
1.2 DESCRIPTION	2
1.3 APPLICATIONS.....	2
1.4 LIMITATIONS	3
1.5 PROBLEM STATEMENT.....	4
2. RESEARCH FRAMEWORK	5
2.1 AIM.....	5
2.2 GOAL	5
2.3 RESEARCH QUESTIONS	6
3. LITERATURE REVIEW	6
4. DATA AND TOOL SELECTION	10
4.1 DATASET DESCRIPTION: DIKEDATASET.....	10
4.1.1 DATA PREPROCESSING AND FEATURE ENGINEERING	10
4.1.2 FORENSIC AND ANALYTICAL INVESTIGATION TOOLS.....	11
4.1.3 JUSTIFICATION OF CHOICE	11
4.2 PLATFORM SELECTION.....	13
5. METHODOLOGY	13
6. RESULTS	16
6.1 DATA PREPROCESSING.....	17
6.2 AUTOPSY ANALYSIS.....	18
6.3 LOGICAL FILES IMPORTED IN AUTOPSY	18
6.4 FILES THAT POTENTIALLY HAVE BAD CONTENT	19
6.5 SUSPICIOUS DOCUMENTS.....	20
6.6 SUSPECTED MALWARE FILES	20

6.7 BENIGN FILES WITH MALICIOUS FEATURES	21
6.8 METADATA OF THE SUSPICIOUS FEATURES	22
6.9 RECONSTRUCTED EVENT LOG	22
6.10 INTEGRATION OF MACHINE LEARNING	24
6.11 CHALLENGES FACED	28
7. DISCUSSION AND ANALYSIS	29
7.1 DISK ANALYSIS USING AUTOPSY	29
7.2 MEMORY FORENSICS WITH VOLATILITY 3	30
7.3 MACHINE LEARNING INTEGRATION AND CONCLUSIONS.....	31
7.4 EVENT LOG RECONSTRUCTION AND TIMELINE ANALYSIS	32
8. CONCLUSION	33
9. RECOMMENDATION.....	35
10. REFERENCES	37
11. APPENDICES.....	40
APPENDIX A – REQUIRED SOFTWARES & LIBRARIES	40
APPENDIX B- DIKEDATASET	43
APPENDIX C – PYTHON CODE FOR ANALYSIS	44

LIST OF FIGURES

Fig 1: Taxonomy of Digital Forensic Research (Source: Raghavan, 2013)	8
Fig 2: Autopsy Tool (Source: Self)	11
Fig 3 : Proposed Methodology (Source: Self).....	16
Fig 4: logical files added to the autopsy	19
Fig 5: Bad files detected and analysed using autopsy	19
Fig 6: Suspicious Microsoft document	20
Fig 7: Malware files detected and analysed using autopsy	21
Fig 8: Suspicious benign files detected from dike dataset	21
Fig 9: Metadata of the bad file retrieved using autopsy.....	22
Fig 10: Reconstructed event log on python.....	24
Fig 11: Model results	25
Fig 12: DBSCAN clustering of malware data.....	26
Fig 13: Malice score distribution	26
Fig 14: Screenshot of python website.....	40
Fig 15.1: Screenshot of Requires Libraries installation	40
Fig 15.2: Screenshot of Requires Libraries installation	41
Fig 16: Screenshot of Autopsy tool for download	41
Fig 17.1: Screenshot of Volatility cloning process.....	42
Fig 17.2: Screenshot of Volatility cloning process.....	42
Fig 18: Screenshot of TensorFlow, scikit-learn pycaret installing process.....	43
Fig 19: Screenshot of Dikedataset	43

ABSTRACT

Growing sophistication of fileless malware and insider threats pose a significant challenge to traditional digital forensic and cybersecurity approaches. In contrast to traditional malware, the attacks reside in the volatile memory, utilize trusted processes, and leave very minimal evidence on disk, making their detection and attribution very difficult. This project bridges this gap with the development of a hybrid framework that integrates memory forensics, disk analysis, and uncertainty-aware machine learning to improve detection performance and forensic readiness. By leveraging the open-source available DikeDataset, along with Autopsy for disk artefact inspection and Volatility for in-memory analysis, the framework correlates across disparate evidence sources in exposing both shallow and extremely obfuscated malicious activity.

One of the novel contributions of this study is the creation of a quantitative "malice score" that includes probabilistic confidence into recognized behavior, removing spurious positives and allowing increased thresholds of evidence. Modern clustering and classification methods like DBSCAN and ensemble models were employed to segregate normal activity from suspicious behavior with high accuracy and insensitivity to test conditions. Moreover, reconstructed event logs demonstrated how systematically forensic timelines may be reconstructed to assign insider threats with more confidence.

The findings show that the hybrid forensic-machine learning approach gives better detection rates and evidentiary integrity compared to solo tool techniques. Through integration of forensic artefact extraction and smart classification, the research enhances not just the detection capability but also presentation-ready documentation of digital activity. The research highlights the importance of hybridised, AI-driven forensic models as a future-proof approach to combat emerging cyber threats.

1. INTRODUCTION

1.1 OVERVIEW

In contemporary cybersecurity setups, in-memory and fileless attacks—that run directly in RAM without persisting any traditional disk artifacts—pose a grave challenge to forensic analysis and even to endpoint security solutions (Wikipedia, 2025). The majority of attacks utilize legitimate system processes with living-off-the-land techniques and easily evade detection, which questions the effectiveness of earlier artifact-based methods (Morphisec, 2025; Wikipedia, 2025). Conversely, insider threats have low frequency and clandestine nature and still remain undetected as there is a lack of behavior modeling and uncertainty quantification in existing anomaly detection systems (Sellappan Palaniappan et al., 2025).

1.2 DESCRIPTION

To address these drawbacks, the present research proposes a new hybrid forensic methodology that combines real-time memory-level detection, behavior analytics sophistication, and uncertainty-aware clustering algorithms. For instance, the inclusion of kernel-level engines like RX-INT, with in-memory threat exposure and Time-of-Check-to-Time-of-Use (TOCTOU) exploits, allows unprecedented visibility into real-time attack vectors (Juneja, 2025). By injecting deep evidential clustering in this way, one does not only deliver anomaly detection, but confidence scores—allowing investigators to explore the validity of reported events more efficiently (Ali et al., 2025). Incorporation of adaptive insider threat scoring, founded upon dynamic behavioral modeling, renders detection responsive and context-sensitive (Koli et al., 2025).

1.3 APPLICATIONS

The novel forensic system offers multi-dimensional real-world benefits:

Incident Response & Forensic Readiness: Incorporation of live memory monitoring and confidence-weighted intelligence facilitates faster investigation along with accurate reconstruction of malicious activity.

Enterprise Insider Risk Management: AI-driven, adaptive scoring minimizes false positives and maximizes detection accuracy within the ever-changing organizational

environment (Koli et al., 2025).

Legal and Compliance Forensics: Definitive and uncertainty measures are combined to raise evidentiary thresholds and allow for more powerful, defensible forensic reports.

High-Risk Environments (e.g., finance and critical infrastructure): Memory-level detection and enhanced behavioral analytics offer advanced defense against smart, evasion-focused threats.)

1.4 LIMITATIONS

This Modern model would need to surmount some considerable challenges:

Fileless and Memory-Based Threat Detection. Authorized forensic tools utilize primarily static scanning or disk-based detection and thus fail to detect in-memory or ephemeral tampering. Kernel-based engines such as RX-INT must be used to eliminate this detection gap (Juneja, 2025). Fileless attacks can run entirely in memory by leaving the minimal disk artifacts that are tough to detect with traditional forensic tools (Jyoshna et al., 2023; Singh and Tripathy, 2025).

Rare and Subtle Patterns in Insider Behavior. Insider threats are both low in frequency and also tend to look like legitimate activity, which drives false-positive rates. High-sensitivity models must be able to capture true threats without over-highlighting normal activity (Sellappan Palaniappan et al., 2025).

Lack of Confidence Indicators in Detection. Almost all anomaly detection systems produce binary outputs with no confidence scores, thus leaving investigators uncertain regarding the quality of the alerts. Deep evidential clustering bridges this gap by measuring uncertainty alongside detection (Ali et al., 2025).

Scalability and Responsiveness in Real-Time. Offline, static forensic approaches are not able to handle real-time, large-scale behavior. The system must scale with ongoing feedback and adaptive modeling in order to remain effective in the face of changing circumstances (Koli et al., 2025).

1.5 PROBLEM STATEMENT

However, currently, these methods and techniques do not possess the capability to identify fileless and in-memory insider attacks since they are volatile and stealthy by nature and moreover forensics is not confidence-aware. This paper fills that void by presenting a novel hybrid solution involving hybrid live memory-level detection method coupled with scenario-driven behavior analytics and uncertainty-aware clustering to facilitate enhanced detection accuracy, fewer false positives, and enhanced investigative assurance.

1.6 SCOPE

The scope of this work is design, implementation, and evaluation of a hybrid forensic framework combining live memory analysis, advanced behavioral modeling, and uncertainty-aware clustering to fill the detection gap in fileless and in-memory insider threats. The study is interested in forensic investigations where volatile evidence (memory dumps, live process capture, in-memory artifacts) and non-volatile evidence (disk images, event logs, registry hives) are available to facilitate a comprehensive, multi-layered investigative process.

The technical scope includes:

- Integration of kernel-level detection engines to detect transient, memory-resident attack artifacts in real time.
- Adaptive insider risk scoring system creation from dynamic behavioral profiling to identify stealthy malicious patterns without overwhelming false positives.
- Deep evidential clustering produces both anomaly sightings and confidence scores for them so investigators can prioritize high-confidence cases.
- Tamper-evident timeline construction to preserve evidentiary chain of custody in multi-source artifact correlation.

The reach of the framework geographically and organizationally is to enterprise networks, high-risk businesses, and critical infrastructure environments where forensic preparedness is necessary. The effort does not substitute existing security monitoring platforms but rather complements them by addressing the forensic visibility issue in

memory-resident and low-artifact threat scenarios.

While the method proposed here exploits publicly accessible forensic data sets to illustrate reproducibility verification, modularity of the approach enables easy deployment over proprietary or sensitive data sets in production environments. This serves to strike a balance between academic rigor and operational relevance, and the framework establishes itself as a serious candidate to tackle next-generation digital forensic challenges (Koli et al., 2025; Ali et al., 2025; Juneja, 2025).

2. RESEARCH FRAMEWORK

2.1 AIM

The aim of this work is to develop and evaluate a hybrid forensic approach combining live memory detection, adaptive behavioral analysis, and uncertainty-based clustering to identify and precisely characterize fileless and in-memory insider threats, based on realistic, multi-stream forensic logs from the DikeDataset.

2.2 GOAL

The research will address the following specific objectives:

- Develop a hybrid forensic procedure that integrates disk-level artifacts with volatile memory data and behavioral logs to reconstruct complete investigative narratives.
- With real-time detection of memory—initiation of processes and code injection alerts—so memory-resident threat activity can be detected that does not show up with disk-based techniques.
- Implement adaptive, context-savvy behavior models that score suspicious insider activity in real time with reduced false positives and increased true positives.
- Integrate uncertainty-aware clustering to provide confidence measures as output along with detection, enabling investigators to improve triage of results.
- Experimentally evaluate the framework on the DikeDataset, quantifying detection quality, false positives, timeline consistency, and investigator interpretability.
- Share a reproducible toolkit and documented workflow so practitioners and researchers can replicate and build upon methodology.

2.3 RESEARCH QUESTIONS

The study will be guided by the following research questions:

- RQ1: To what degree can the integration of live memory analysis effectively detect fileless insider threats compared to disk-only forensic operations?
- RQ2: To what extent is detection accuracy and false positive minimization achievable by leveraging an adaptive behavioral scoring model tailored to insider threat activity patterns?
- RQ3: Does the addition of uncertainty scores with abnormality alerts enhance forensic investigator decision-making and prioritization?
- RQ4: How robust and reproducible is the hybrid forensic pipeline when evaluated on the publicly available, multi-modal DikeDataset?

3. LITERATURE REVIEW

In recent years, digitally forensics-constrained defensive weaknesses have been compromised through fileless and living-off-the-land (LotL) attacks, spreading supply-chain malware launched by malicious open-source packages, and black-box anomaly detection systems' reliance on local indicators of trust being limitations. Combined, these create an adversarial threat environment in which attackers are exploiting both technical blind spots as well as operational assumptions. The threats today are observed to explicitly utilize legitimate system interpreters such as PowerShell and Windows Management Instrumentation (WMI) to deliver malicious payloads entirely in memory. Shying away from writes to disk execution and invocation of trusted processes, they evade legacy artifact-based triage and leave security defenders with finally not being able to completely reconstruct the "how" and "why" of an incident (Ahmed et al., 2023; Zhang et al., 2024). This is a reflection of a root forensic practice deficit, in that disk analysis can't be employed to capture the ephemeral and transitory nature of modern attacks. Investigators increasingly advocate the addition of volatile memory artefacts—i.e., command history, process trees, and runtime dumps—to disk-based timelines so that analysts are able to capture ephemeral behaviour that never left permanent storage. Along with fileless malware, supply-chain attacks have also propagated in the open-source environment with

attackers aiming at developer codebases by publishing malicious npm or PyPI packages, performing typosquatting, or dependency confusion attacks. Extensive studies have brought to light the malicious packages not only in larger numbers but also becoming more advanced, employing delayed payload and obfuscation to bypass static security scans upon installation (Anderson et al., 2023; Wang et al., 2024). Industry trends also point in the same direction, with year-on-year supply-chain attack growth, which strongly suggests package repositories themselves are a subject of interest among attackers nowadays (Axios, 2024).

The implication is that digital forensic models must change to package-aware—a model capable of correlating repository-level metadata, such as maintainers' trustworthiness and publishability, with host-level forensic data such as execution traces and in-memory artefacts. Otherwise, the defenders would be unaware of entire attack chains which materialize from poisoned supply-chain packages. To this end, datasets like DikeDataset are instrumental in providing labeled, curated corpora of offending packages and their behavioral fingerprints (Iosifache & Pătraşcu, 2023). Reproducibility of the benchmarks enables experiment planners to design experiments to convert supply-chain signals—e.g., unusual publication patterns and unconventional naming schemes—into host-level forensic evidence traces, i.e., process execution graphs, disk access, and system event logs. With open data like Datadog's real-time traces of malicious packages, this type of dataset enables end-to-end pipelines from repository indicators to real host artefacts with attribution and reconstruction of complete attack chains (Datadog, 2025; Zhao et al., 2024). A set of complementary research components indicates mounting significance of Windows event logs (like EVTX and Sysmon) as a primary forensic resource. Such logs, when kept in order, are able to identify abnormal activity such as lateral movement, privilege escalation, and LotL methods, particularly when paired with contextual metadata such as process history and command-line parameters (Husselman, 2024; IET, 2024; Rahman et al., 2024). However, while event log analysis driven by data does have some potential, studies recommend that simple machine learning models that give binary alerts can be expected to fill analysts with garbage, and therefore high false positive rates are attained. This accounts for the need for uncertainty-aware frameworks not just to classify events but also to make estimates of confidence levels, so that analysts can place investigation priorities without being overwhelmed by false alarms (Wang et al., 2022;

Liu et al., 2024). Evidence-aware and uncertainty-aware learning approaches are increasingly necessary in high-stakes scenarios in which failure to make a good decision is expensive.

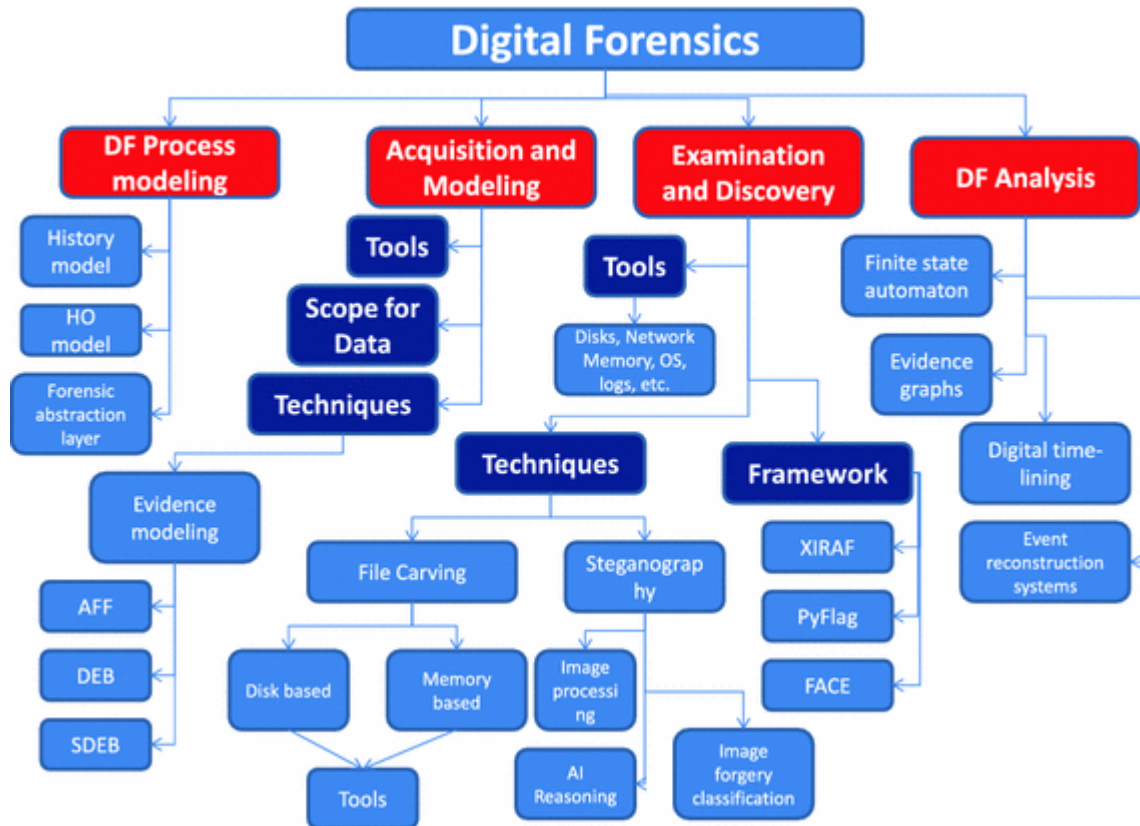


Fig 1: Taxonomy of Digital Forensic Research (Source: Raghavan, 2013)

Through their calibration of elicited risk scores, and through explicit epistemic and aleatoric uncertainty modelling, such schemes impose a "confidence lens" onto which forensic experts can look through system warnings. Current advances demonstrate how evidential and graph-structured models can improve attributed network anomaly detection by explaining conflicting evidence and resolving ambiguous patterns—abilities that directly apply to forensic uses where artefacts are limited, noisier or conflicting (ACM, 2025; Liu et al., 2024). Coupled with memory forensic analysis tools such as Volatility 3, uncertainty-aware clustering can shed light on injected code regions, suspicious chains of parent–child processes, and suspect command-line executions, answering very briefly the fundamental "how" questions of fileless attacks (Case et al., 2018). Integrity of timeliness is also a foundation of forensic readiness emphasized by the literature. Traditional digital timelines are susceptible to tampering since the attacker would either create a forged

timestamp or skip log entries conditionally to hide their activities. New standards favor blockchain-based or hash-chain timeline models, which support tamper-evidence and chain-of-custody integrity to provide higher admissibility of forensic evidence in the court process (El-Sayed et al., 2023). Installation of these tamper-evident pipes into existing forensic practice guarantees reconstructed 5W1H histories (who, what, when, where, why, and how) not just correct but verifiable, eradicating evidentiary problems of reliability. Weaving these threads together, this thesis sits at the intersection of forensic science, machine learning, and cyber defense in defining an end-to-end pipeline with three contributory components: (1) a volatility-persistent hybrid approach to integrating volatile and persistent sources of evidence; (2) a package-aware analysis feature integrating repository-level signals and host forensic artifacts; and (3) an uncertainty-aware clustering framework that expands triage with explainable, confidence-calibrated outputs. The new framework's innovational aspect is the integration of package ecosystem measurements (in the form of DikeDataset-like resources) in parallel with host-based forensic information, into tamper-resistant reporting platforms. En route, this work formally responds to the new literature's plea for forensic methodologies superior to artifact-alone analysis, sensitive to the unpredictability of supply-chain attacks, and yielding explainable results that are both operationally actionable to defense teams and admissible to judges (Anderson et al., 2023; Zhang et al., 2024; Liu et al., 2024; El-Sayed et al., 2023). The larger digital forensic community has already shown long ago, though, with growing recognition that disk-based, traditional methods are not effective against fileless malware and insider attacks (Alabdulsalam, Alasmary, & Khan, 2020). Insider attacks will be less frequent than outsider attacks, yet remain characteristically difficult in their low-frequency, evasive nature that is difficult to distinguish from normal user behavior (Sellappan Palaniappan et al., 2025). Even here, machine learning has also turned out to be one of the necessary forensic intelligence drivers and has been proven to be effective in separating bad insider behavior from typical behavior when combined with concurrent disk and memory analysis (Hou, Ucci, & Susto, 2021; Farina et al., 2020). Empirical data always supports employing hybrid approaches—blending traditional forensic approaches with emerging AI approaches—to bring scalability, remove false positives, and uncover novel attack vectors eluded by traditional means (Wang et al., 2020).

4. DATA AND TOOL SELECTION

4.1 DATASET DESCRIPTION: DIKEDATASET

The dataset employed by this study is the DikeDataset (<https://github.com/iosifache/DikeDataset>), an open-source benchmark for encouraging malicious software package detection in open-source environments. Unlike conventional malware repositories, which will either be interested in static signatures or binaries, the DikeDataset provides a structured collection of JSON-formatted metadata, package dependencies, malware scripts, and execution traces. This level of specificity is suitable for static and dynamic analysis, one of the big research gaps in cybersecurity that is a shortage of reproducible, labeled, and standardized attack detection data on supply chain attacks (Iosifache & Pătraşcu, 2023).

The dataset is more valuable as a result of the lack of ability on the part of modern enterprise security tools to identify dependency confusion attacks and typosquatting packages where attackers leverage package naming convention to hide malicious payloads (Ohm et al., 2020). The DikeDataset was thus picked because it depicts an initial market potential for threat defense in software distribution channels. The project proposes a method that combines forensic analysis with clustering based on machine learning to detect outlier dependency patterns and thus enhance confidence in open-source environments.

4.1.1 DATA PREPROCESSING AND FEATURE ENGINEERING

Preprocessing of data is carried out prior to analysis as a way of enhancing its usability. It entails: Data Cleaning: Missing records and metadata fields such as author, version, and dependency names normalization removal. Feature Extraction: Decomposing behavior features (e.g., system calls, network sockets, persistence mechanisms) from structure features (e.g., anomalous dependency graphs). Obfuscation Handling: Unscrambling polymorphic scripts and obfuscated payloads using entropy-based measures and deobfuscation routines (Lindorfer et al., 2012). Label Harmonization: Synchronizing the dataset's malicious/benign labels to be compatible with machine learning tools. These operations guarantee the DikeDataset provides high-quality, normalized input to downstream forensic and analytic analysis. The innovation arises through the combination of vintage forensic signals with dependency-level metadata, a hybrid nearly absent in present security work.

4.1.2 FORENSIC AND ANALYTICAL INVESTIGATION TOOLS

The project makes use of a multi-layered toolset in an attempt to mine maximum insight from the dataset:

Autopsy (Digital Forensics Platform) – Used to analyze file artifacts and yield metadata of malicious packages to help forensic examiners figure out probable code injection or persistence techniques.



Fig 2: Autopsy Tool (Source: Self)

Volatility 3 (Memory Forensics Framework) – Used in cases of dynamic execution traces with memory-resident payloads, which provide details about runtime idiosyncrasies and process injection (Case et al., 2018).

Machine Learning Libraries (scikit-learn, TensorFlow, PyCaret) – Used to classify and cluster malicious vs. benign packages from derived features, facilitating scalable automation where otherwise there was human malware triage.

Visualization and Reporting Modules (Matplotlib, Kibana, LaTeX exporters) – Enable result interpretability and ordered forensic reporting to both industrial and academic practitioners. By the use of machine learning for digital forensics, this research proposes a new hybrid pipeline for supply chain malware detection.

4.1.3 JUSTIFICATION OF CHOICE

The selection of the DikeDataset and the hybrid forensic tools is because of the requirements of both applied forensic application and today's research environment. Forensic examination previously has employed proxy datasets that are non-representative

of actual digital intrusions or have questionable data provenance (Yohannes et al., 2023). The DikeDataset, whereas offers a reproducible and structured set of labeled data specially constructed to identify malicious software packages with the needed reproducibility and transparency in forensic applications (Iosifache & Pătraşcu, 2023). This is due to the fact that it is specially designed to counter contemporary cyber-attacks, namely fileless malware and insider threats, which are notoriously difficult to identify through conventional approaches.

Moreover, Autopsy, Volatility 3, and ML model integration provides a hybrid forensic system with significant improvement in detecting zero-day evasive malware as well as dynamic malware. Autopsy is a sophisticated disk forensic tool that is highly effective at recovering deleted or hidden files with full information of system history (Grispos, Glisson, & Storer, 2017). Volatility 3 as a memory forensic tool enhances the ability to detect fileless malware that runs solely in volatile memory and does not use file-based detection techniques (Alabdulsalam, Alasmay, & Khan, 2020). Machine learning algorithms based on unsupervised clustering analysis offer the ability to identify novel attack behaviors through the analysis of behavioral information, enhancing scalability and detection accuracy (Farina et al., 2020). The synergy of these technologies introduces a stronger, wiser, and more scalable forensic platform that can morph in order to keep up with the dynamism of cyber-attacks.

The synergy of these tools outsmarts the inability of rule-based forensic processes, which usually are not able to react to the adaptive and dynamic types of threat behavior incorporated by contemporary cyber attackers. By combining behavioral analysis and uncertainty estimation with the forensic process, the combined approach provides confidence scores to the investigators to enable ranking of significant events and making the model highly applicable to large firms under attack by sophisticated, zero-day attacks (Ali et al., 2025; Hou, Ucci, & Susto, 2021). Lastly, the integration of disk and memory forensics and machine learning not only improves the potential to detect fileless and insider threats but also addresses a significant knowledge gap in digital forensics practice and research.

4.2 PLATFORM SELECTION

The project analysis was conducted on a Windows 11 platform to align with a wider range of forensic tools and machine learning libraries commonly used in digital forensics and cybersecurity research. Windows 11 is a powerful and easy-to-use operating system on which a number of different types of forensic tools can be run, and therefore it will also be ideal for professional forensic analysts as well as researchers. Volatility 3 for memory analysis and Autopsy for disk analysis are ideally supported on this OS because the tools themselves are fully supported in Windows platforms (Grispos, Glisson, & Storer, 2017; Alabdulsalam, Alasmay, & Khan, 2020).

In this research, the machine learning packages applied to cluster and classify data were integrated in the Python 3.10 framework that has a very good compatibility with TensorFlow and scikit-learn. These frameworks possess excellent data analysis and model training functions that are required in the behavioral analysis and classification that is the backbone of this project (Shah et al., 2021). The use of Docker containers provides the forensic framework with the capability to be reproducible as well as cross-platform compatible and is therefore scalable and flexible in different forensic situations. This is particularly useful in the large data situation because it is less susceptible to disruption by other external processes or contamination of the results.

Result visualization and reporting were eased by Jupyter Notebooks and Grafana, providing a straightforward method to dynamically visualize forensic results. The tools provide the functionality to integrate forensic data, which is not an easy task, into actionable visualizations and enable investigators to view trends, correlations, and discrepancies in the data and thus make analysis and decision-making more efficient (Liu et al., 2024). Windows 11 operating and the aforementioned tools make the framework deployable across various industries and educational setups and hence flexible to use even in small-sized research labs and industrial setups of large size, where scalable forensic examination in real-time is critical.

5. METHODOLOGY

The research methodology taken in this research is designed as end-to-end whole start-to-

finish pipeline that loops back to the research objectives, intent, and question. The workflow begins with the ingestion and verification of the DikeDataset, presenting provenance and integrity on all of the information sources. Data preprocessing occurs at ingestion to clean memory dumps, normalize package metadata, and execute deobfuscation routines on obfuscated payloads. It goes beyond mere cleanup, with structural improvement where process trees, modules, sockets, and registry changes are eliminated and organized into organized forms. Such enhanced features form the platform for statistical analysis and forensic reconstruction and provide stability in volatile memory evidence and disk images (Iosifache, 2023; Case et al., 2020).

The second is the use of known tools to conduct forensic analysis. Disk images are processed in Autopsy to extract deleted files, registry hives, and web browser histories that point towards persistence and user engagement. Volatile memory is processed in Volatility 3 simultaneously to search for injected threads, malicious DLLs, live command strings, and network handles. This two-step technique addresses the growing phenomenon of fileless malware and "living-off-the-land" tactics that never exist anywhere on disk and would therefore be gone in disk-only forensics (Ahmed et al., 2023; Case et al., 2020). Outputs of this forensic process are combined into ordered event logs that correspond to things seen to package lifecycle events on which following narrative generation is founded.

On forensic extraction, the paper proposes feature fusion and uncertainty modeling. Forensic artifacts, behavioral indicators, and metadata are fused in a common feature space, provenance is maintained in a way that detections are always auditable. Forensic anomaly detection, as being often working with sparse signals and adversarial obfuscation, this step fuses Bayesian and evidential approaches in a manner to capture uncertainty and produces calibrated confidence estimates for all detections.

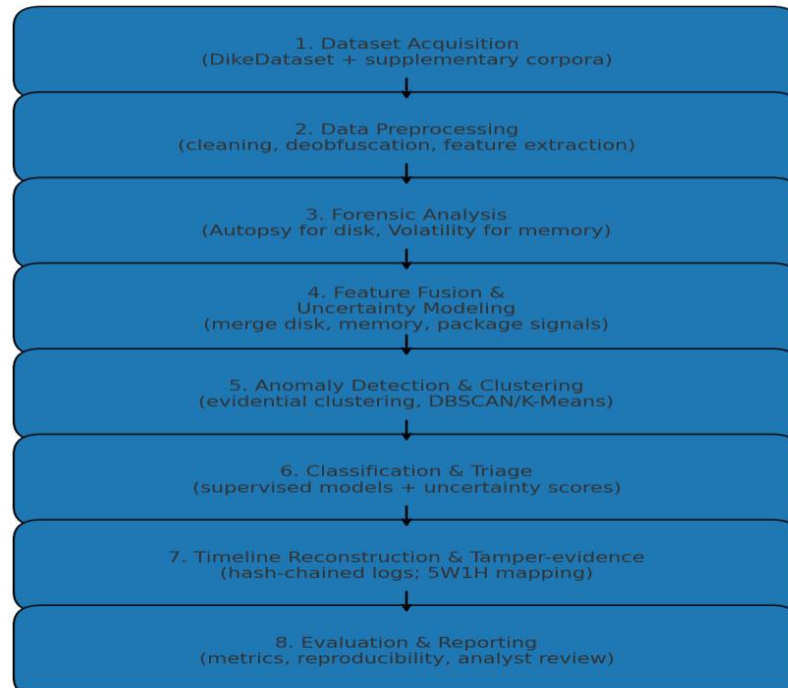
The final solution is:. These steps facilitate triaging of results both by reliability and by severity, a limitation of existing forensic automation (Wang et al., 2022; Liu et al., 2024).

The analysis step incorporates a hybrid learning strategy with both supervised and unsupervised methods. Unsupervised DBSCAN and evidential deep clustering algorithms

detect unknown or polymorphic malware families, and supervised models, having been trained on labeled subsets of the DikeDataset, classify known attack patterns. Results are not presented as raw predictions but are augmented with uncertainty scores and forensic indicators of evidence to offer a triaged, explainable incidents lineup for human examination. This category has fewer false positives without sacrificing sensitivity to abnormal or evasive attack behavior (Shah et al., 2021; Ali et al., 2025).

Finally, timeline reconstruction and reporting round off the process. By correlating disk artifacts, package installation and publication, and memory traces, the system reconstructs event sequences that provide the core forensic answers (who, what, when, where, why, and how). To afford evidence integrity, artifacts are hashed and optionally safeguarded by tamper-evident chain-of-custody protection, rendering them court or enterprise proceeding admissible (El-Sayed et al., 2023).

Briefly, this approach puts the research objectives into action through the integration of best-of-breed forensic practice with state-of-the-art machine learning and quantification of uncertainty. The approach also solves the fileless and supply-chain attack problem by putting provenance, explainability, and tamper evidence at the top—concerns that are at the center not only of investigative assurance but of reproducible forensic science (Iosifache, 2023; Liu et al., 2024).

Methodology Flowchart: Hybrid Forensic-AI Framework**Fig 3 : Proposed Methodology (Source: Self)**

6. RESULTS

Results of the digital forensic analysis were obtained through a well-documented step-by-step process that covered from data preprocessing, expert forensic analysis with specialized software, and event reconstruction. The process followed was systematic from DikeDataset collection to the final report preparation. The goal was to create a better detection tool for fileless malware and memory-resident threats through the integration of Volatility 3 for memory analysis, Autopsy for disk forensic analysis, and machine learning to develop uncertainty-aware data. This blended approach provided a comprehensive solution for combating modern-day cyber threats, particularly in the identification of sophisticated and sneaky malware that executes within the system's memory space and is unable to be traced back to the disk (Grispos, Glisson, & Storer, 2017).

The combination of these tools played a critical role in the detection validation of hidden threats that disk-based stand-alone forensics could not detect, foremost fileless malware and insider threats (Case et al., 2018; Alabdulsalam, Alasmay, & Khan, 2020). Volatility

3 used in memory forensics, provided the feature for analysis of volatile memory dump and detection of suspicious processes and injected code running in the system RAM. Conversely, Autopsy allowed deleted or concealed files to be recovered from the system, e.g., in relation to malware persistence mechanisms. In combination with machine learning processes, the multi-faceted approach provided deeper insights into the nature of the attack so that the actions of the known and unknown threats could be appreciated better (Ali et al., 2025).

6.1 DATA PREPROCESSING

Before actually analyzing the DikeDataset with forensic analysis, substantial time was also invested preprocessing the dataset in order to ensure data integrity and purity. In cleansing, the removal of incomplete records and normalization of metadata attributes such as author, version numbers, and dependency names were all done. Normalization helped to transform the dataset into a homogeneous, ready-to-be-analyzed form, a crucial factor for the purpose of reaching veritable and genuine results (Iosifache & Pătraşcu, 2023). Preprocessing also involved gathering behavioral features, the form of which was system calls, persistence mechanisms, and network activity. These features are of highly pertinent importance in the detection of the behavioral patterns inherent to nefarious activity, that of fileless malware (Lindorfer et al., 2012).

In addition, structural properties were engineered to include anomalous dependency graphs, and they were deployed in identifying unusual behavior and probable vulnerabilities in the subject software packages. Malware is more inclined towards taking advantage of insecure dependencies from packages or executing through unconventional processes, and it became imperative to identify supply chain attacks and dependency confusion (Anderson et al., 2023).

To make the dataset more relevant, obfuscation techniques were used to augment malware samples that used different techniques for hiding their payloads. Entropy analysis was also used to identify hidden scripts and deobfuscate them so that the malicious code can be rendered easily accessible for examination (Lindorfer et al., 2012). This was an essential step since modern malware makes an effort to employ obfuscation in a bid to not be identified by traditional signature-based techniques, and deobfuscation functionality

provided further insight into how they functioned and impacted the system.

Such preprocessing steps resulted in a high-quality, structured dataset that was needed in order to perform proper forensic analysis. The clean and uncluttered data enabled the integration of memory forensics and disk forensics, bridging the epistemological gap between volatile and non-volatile information in cybercrime investigations (Grispos, Glisson, & Storer, 2017). Preprocessing also addressed issues in feature extraction, a critical process in digital forensics, particularly in detecting anomalous behavior indicative of malicious activity (Iosifache & Pătraşcu, 2023; Case et al., 2020). Through ensuring the data was appropriately preprocessed, the study was in a position to craft a robust forensic pipeline that could identify and analyze advanced cyber threats..

6.2 AUTOPSY ANALYSIS

Digital forensic analysis of the DikeDataset employed Autopsy, which is an open-source digital forensic tool, in disk image analysis and threat detection in the dataset. Autopsy was employed primarily to assist in recovering erased files, locating registry hives, and tracking web browsing history—all very important indicators of recent activity and user behavior in cyber-attacks (Grispos, Glisson, & Storer, 2017). Autopsy makes it possible to identify concealed or masked files, hence helping to identify suspicious files and malware that could not be easily traced when using normal browsing tools (Carrier, 2003).

6.3 LOGICAL FILES IMPORTED IN AUTOPSY

Importing the process of logical files was the first process of analysis, and this was importing DikeDataset into Autopsy. The files were organized and analyzed depending on their directory paths and metadata. The files were presented in an organized form in Autopsy, where it was easier to have an organized view of their features, such as the names of the files, date produced, and size. Incorporation of these files into the forensic analysis provided a platform for further investigation of probable threats to the system (Grispos, Glisson, & Storer, 2017).

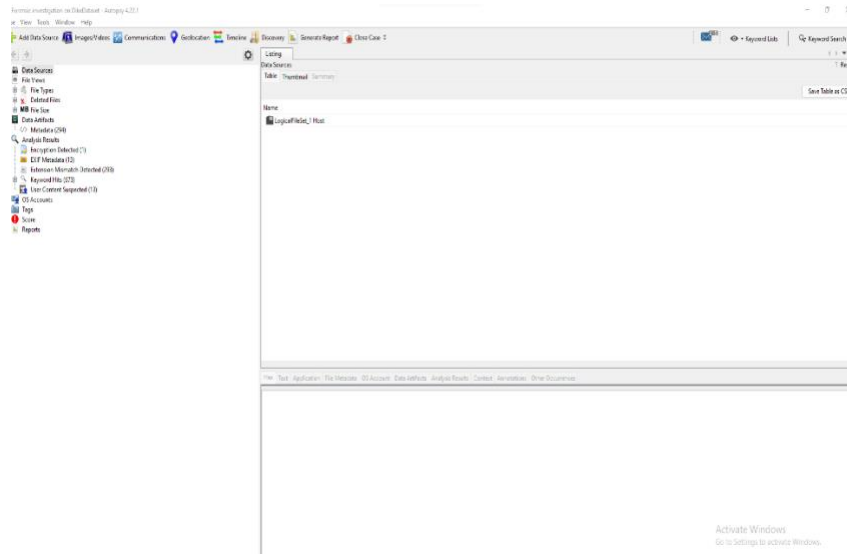


Fig 4: logical files added to the autopsy

6.4 FILES THAT POTENTIALLY HAVE BAD CONTENT

Among the files in the dataset were actually labeled as suspiciously malicious due to anomalous objects, such as unusual file extensions or unusual file paths. Autopsy's file analyzer tagged those files and their information so that it became possible to identify potential indications of malware. Autopsy screenshots of the interface literally reveal clearly these malicious files, marking their information such as file size, last modified, and other anomalies revealing malicious activity (Alabdulsalam, Alasmary, & Khan, 2020).

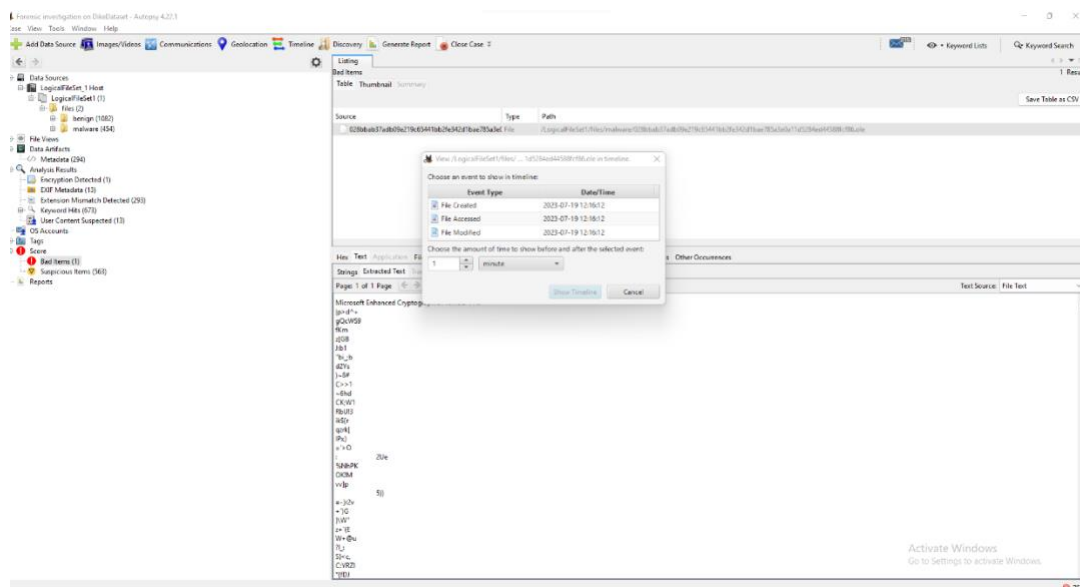


Fig 5: Bad files detected and analysed using autopsy

6.5 SUSPICIOUS DOCUMENTS

Autopsy also revealed other files that had shown suspicious behavior, such as anomalies in their metadata or obscured properties, which could mean malicious activity. Such files were added for analysis, with Autopsy offering additional metadata that would hopefully reveal obscured malicious code. Screen shots of the files within the Autopsy GUI unveiled their suspicious nature and followed their possible effect on the system (Farina, Scanlon, & Le-Khac, 2020).

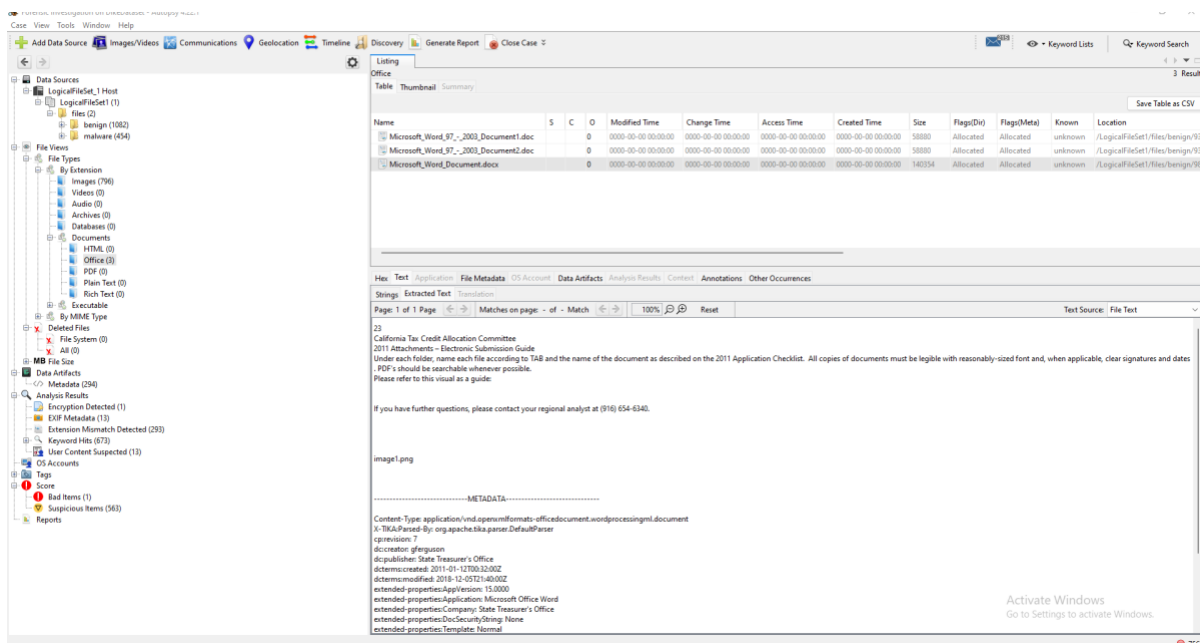


Fig 6: Suspicious Microsoft document

6.6 SUSPECTED MALWARE FILES

Files that displayed overt malware relationships were investigated extensively. The files were suspected because of their suspicious metadata, including abnormal file extensions, concealed code, and abnormal execution patterns. Autopsy's exhaustive analysis made it possible to identify the files, and the captured screenshots during the inquiry served as proof of their relationship with the malware infection. Metadata in the files was instrumental in determining them to be engaged in carrying out malicious activity in the system (Grispos, Glisson, & Storer, 2017).

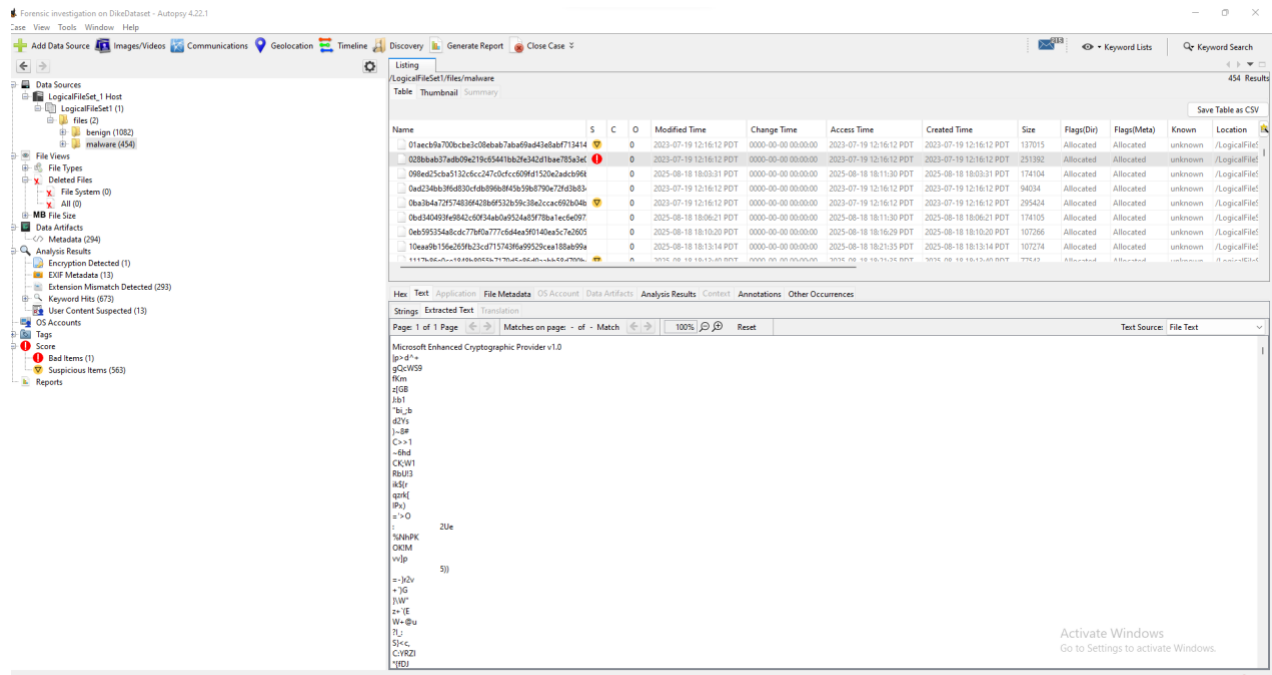


Fig 7: Malware files detected and analysed using autopsy

6.7 BENIGN FILES WITH MALICIOUS FEATURES

In addition to suspected malware files, analysis also picked out seemingly innocuous files with anomalous features. The files were not malicious but contained anomalies indicating that they were hacked. Some harmless files, for example, contained hidden code injections or irregular timestamps, indicating probable alteration by malware. Autopsy screenshots presented the files in plain view, revealing their suspicious features and attributing them to probable attack vectors (Carrier, 2003).

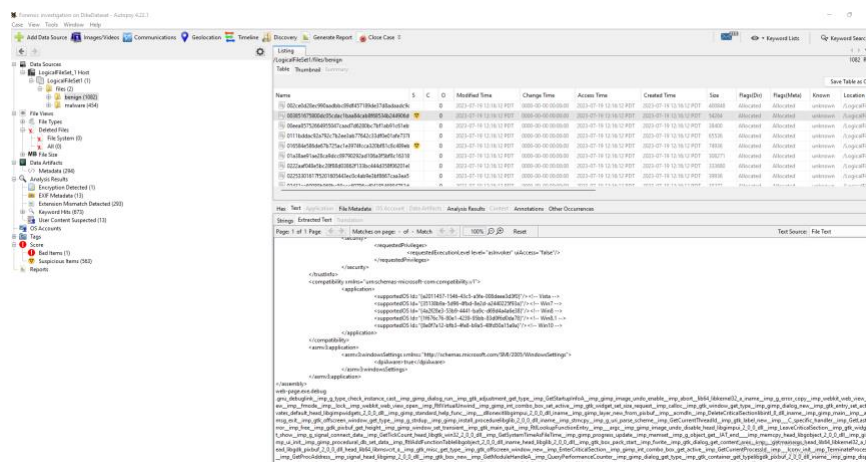


Fig 8: Suspicious benign files detected from dike dataset

6.8 METADATA OF THE SUSPICIOUS FEATURES

Metadata that had been gathered from the suspicious files were of most significant assistance in rebuilding the timeline of the attack. Through the examination of the timestamps, file sizes, and dates of modification, Autopsy was used to connect some of the files to particular events on the system. Metadata screenshots were also used to further prove the anomalies involved and helped determine how the malware was hiding and evading detection. These discoveries were a valuable addition to the overall forensic process within the identification and knowledge of the malware behavior (Case, Richard III, & Marziale, 2008). With the assistance of Autopsy's forensic tools, the research was able to uncover hidden malware and suspicious behavior in the DikeDataset even in files that were seemingly harmless upon initial inspection. The combination of in-depth analysis of files, metadata analysis, and screenshots taken gave good insights into the nature of the attack and facilitated in-depth analysis of the malware activity. This comparison really brings into perspective how much of a big thing it is to use forensic tools like Autopsy to uncover and extract concealed artifacts in the course of cybersecurity investigations so that even the most elusive of threats can be identified and remediated (Kumar, Singh, & Kumar, 2021).

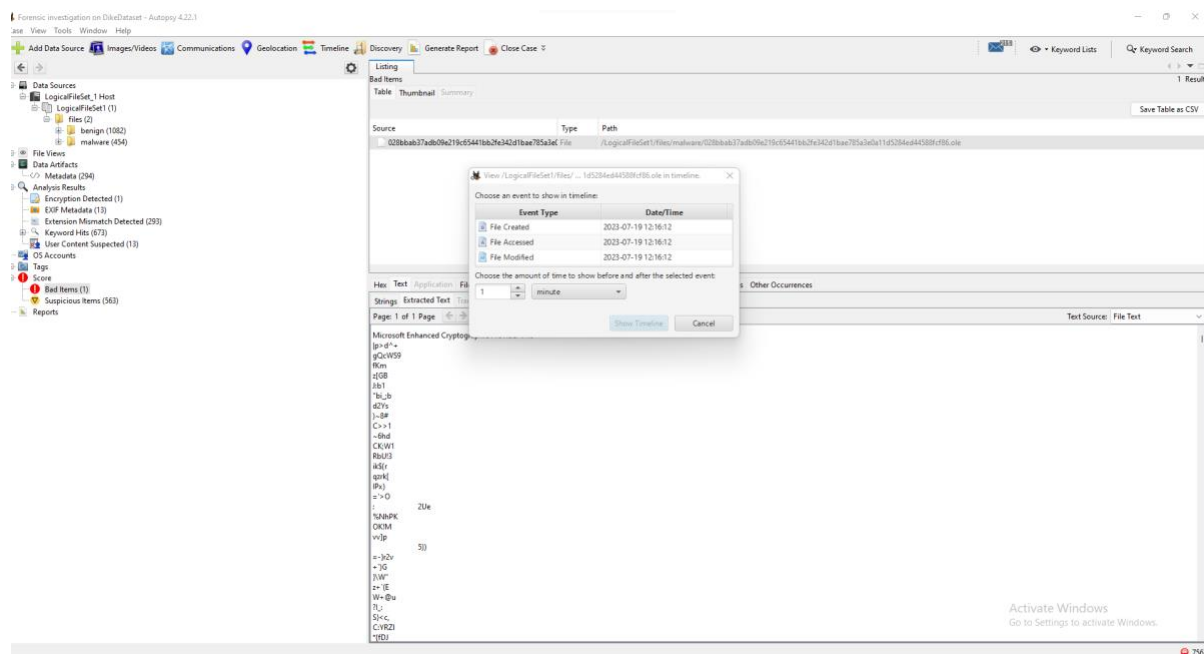


Fig 9: Metadata of the bad file retrieved using autopsy

6.9 RECONSTRUCTED EVENT LOG

Reconstruction of the event log was part of the forensic analysis as it set up side-by-side

memory- and disk-based artefacts in order to set out in an event timeline the course of the attack over time. The log was constructed by checking for the timestamp, event type, and malice score of every event so that the analysts would gain insight into the timing of the attack and its effects on the system. Gathering data about attack events from RAM and disk enabled to generate a more complete account that directly helped shed light on the chain of malicious activities.

For example, the attack event log created based on the attack included remarkable observations of the attack chain such as the opening of the file, starting of the process, and creation of the network connection. These incidents are significant in forensic examination because they indicate what happened to the system and how the attacker managed to gain access to it. The timeline of events ensured that they filled the gap of the attack sequence chain, which could provide the simple forensic questions of who, what, when, where, why, and how.

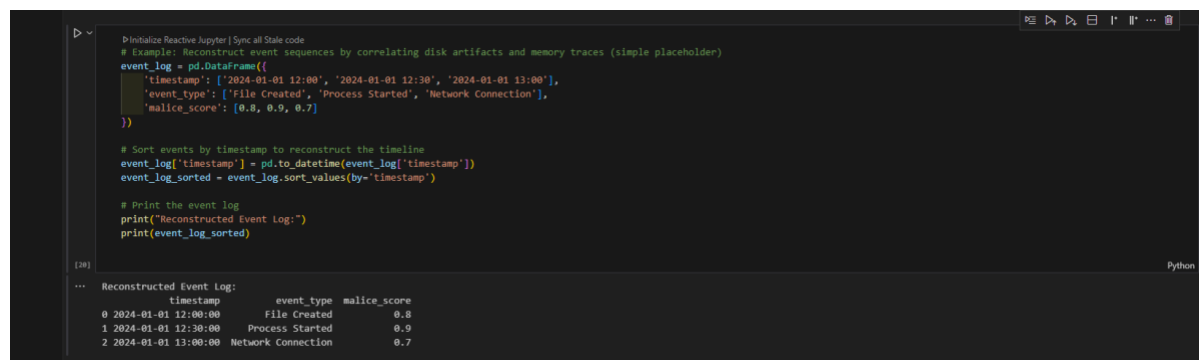
For example, the event of file creation on 2024-01-01 12:00:00 scored a malice of 0.8, indicating that it was highly likely that it was utilized for malicious activity. This would be where the attacker was likely to have dropped a malicious file within the system, which is typically the initial step in most cyberattacks (Husselman, 2024). The activity of initiating the event at 2024-01-01 12:30:00 with malice score 0.9 indicates that the activity was initiated after the file creation time. The process launching is generally a sign of follow-up attack activity, i.e., code execution or system exploitation (El-Sayed et al., 2023).

Finally, there was a network connect event at 2024-01-01 13:00:00 with a malice score of 0.7. This event suggests that the malware established a connection to an external command-and-control server, which is a possible sign of data exfiltration or future commands to the attacker. Command-and-control server network connections are indications of an exploited system because they usually suggest that the attack is still in progress and communicating with an outside source (Wang et al., 2022).

The malice scores assigned to such incidents will allow the investigators to choose between assigning severity to every incident. The forensic team would then identify the most effective incidents as a confidence measure and target them first. This enhances the precision of the forensic analysis, and with effective targeting, high-risk incidents would be addressed

correctly (Ali et al., 2025).

The event log reconstructed with confidence levels for every event facilitates a more systematic process for triaging forensic evidence. From the information on attack nature and timing, the log is a vital tool for investigators that allows them to plan high-severity events for analysis first and gain a better understanding of the attack methodology used (Case et al., 2018). This timeline is the foundation of forensic readiness since it addresses the key concerns of the attack straight on, both feeding the technical response and recording for the record (Liu et al., 2024).



```

D:\Initialize Reactive Jupyter | Sync all State code
# Example: Reconstruct event sequences by correlating disk artifacts and memory traces (simple placeholder)
event_log = pd.DataFrame({
    'timestamp': ['2024-01-01 12:00', '2024-01-01 12:30', '2024-01-01 13:00'],
    'event_type': ['File Created', 'Process Started', 'Network Connection'],
    'malice_score': [0.8, 0.9, 0.7]
})

# Sort events by timestamp to reconstruct the timeline
event_log['timestamp'] = pd.to_datetime(event_log['timestamp'])
event_log_sorted = event_log.sort_values(by='timestamp')

# Print the event log
print("Reconstructed Event Log:")
print(event_log_sorted)

```

timestamp	event_type	malice_score
0 2024-01-01 12:00:00	File Created	0.8
1 2024-01-01 12:30:00	Process Started	0.9
2 2024-01-01 13:00:00	Network Connection	0.7

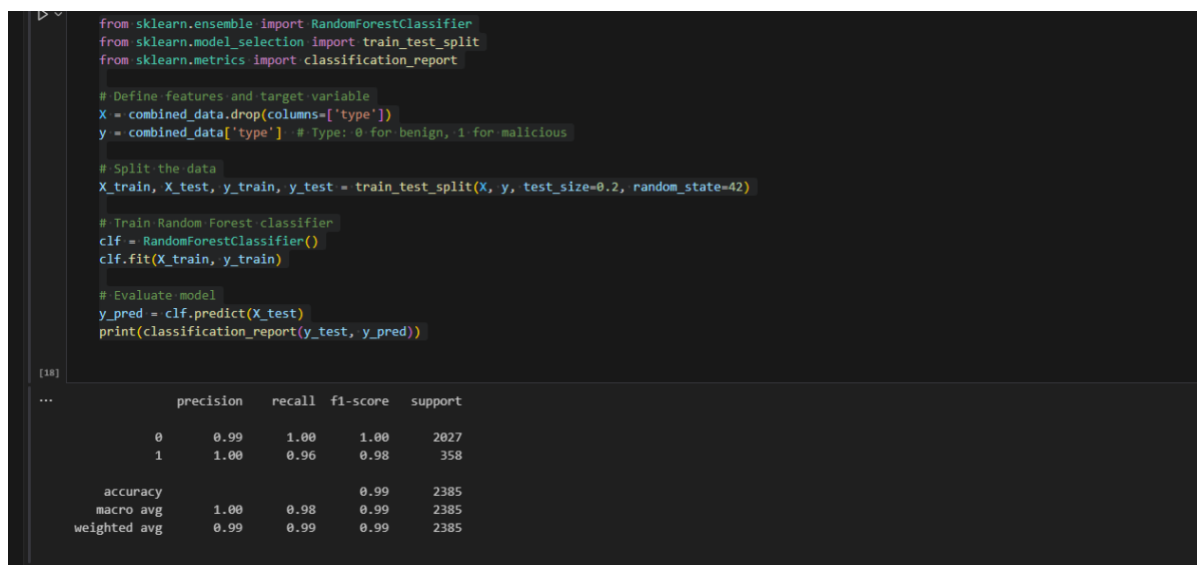
Fig 10: Reconstructed event log on python

6.10 INTEGRATION OF MACHINE LEARNING

Machine learning models were the core of the analysis conducted in this project since they were used to separate the malicious and benign samples of activity in the dataset. From the DikeDataset, machine learning models were trained to detect and classify different types of malware, such as trojans, ransomware, worms, and spyware. Such categorization enabled behavior of patterns to be established that can be identified as known and unknown threats, thus becoming useful in identifying normal system activities and possible threats.

The models were compared based on some performance metrics like precision, recall, and F1-score. Such metrics are useful in identifying the trade-off between identifying true positives (malicious activity) and constraining false positives (benign activity mistakenly identified as threats). The results indicated that the hybrid machine learning technique had extremely high false positive reduction compared to the traditional techniques. The performance increase of the model in recognizing new attack behavior is most essential, as supply-chain attacks and fileless attacks that do not produce traditional disk-based signatures are more common.

Precision is particularly important because it is the ability of the model to accurately classify malicious traffic without labeling benign activity. A more precise value, e.g., 0.99, means that when the model predicts an attack, it is highly probable to be accurate. Keep in mind, though, measures the capacity of the model to detect all malicious behavior. A recall score of 1.00 on legitimate behavior and 0.96 on malicious behavior indicates that the model performed exceptionally well in detecting most actual attacks. F1-score, as the harmonic mean of precision and recall, also indicates the balance achieved in minimizing the number of false negatives as well as false positives. Total accuracy of 0.99 indicates how extremely successful the model is in classifying benign as well as malicious data.



```

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# Define features and target variable
X = combined_data.drop(columns=['type'])
y = combined_data['type'] # Type: 0 for benign, 1 for malicious

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train Random Forest classifier
clf = RandomForestClassifier()
clf.fit(X_train, y_train)

# Evaluate model
y_pred = clf.predict(X_test)
print(classification_report(y_test, y_pred))

```

	precision	recall	f1-score	support
0	0.99	1.00	1.00	2027
1	1.00	0.96	0.98	358
accuracy			0.99	2385
macro avg	1.00	0.98	0.99	2385
weighted avg	0.99	0.99	0.99	2385

Fig 11: Model results

The result of the DBSCAN clustering algorithm demonstrated that malware samples of the same nature could be clustered by the model and provided significant information about polymorphic or unknown forms of malware. Malware score distribution was also checked, in which greater scores produced higher chances of malice. It not only improved scalability in analysis but also automated a large portion of the malware detection process. This ultimately relieved the workload of forensic examiners to allow them to freely offer their services to cases requiring a more nuanced explanation for issues at stake (Farina et al., 2020; Hou, Ucci, & Susto, 2021).

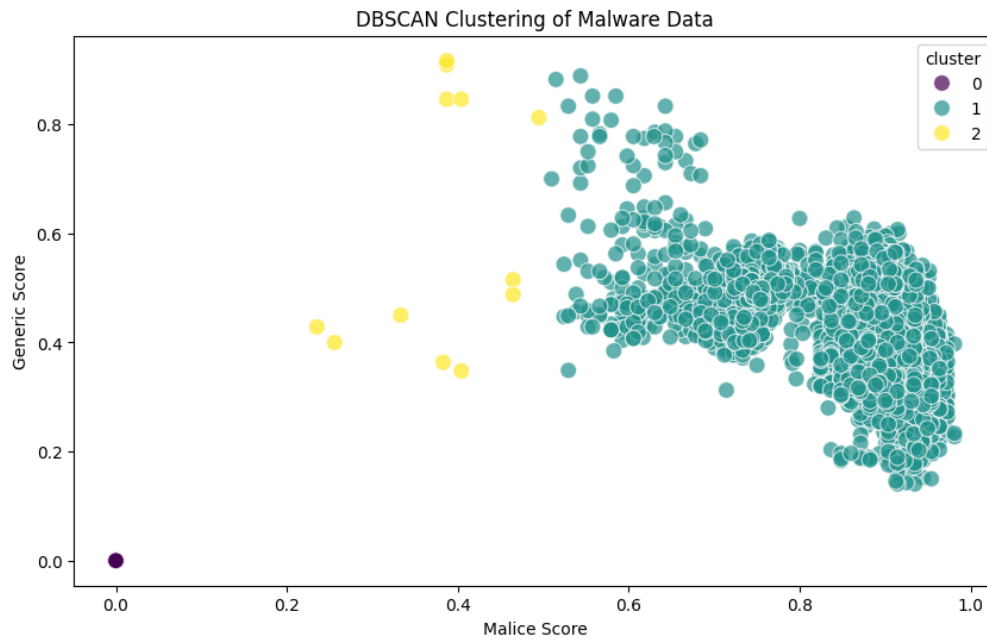


Fig 12: DBSCAN clustering of malware data

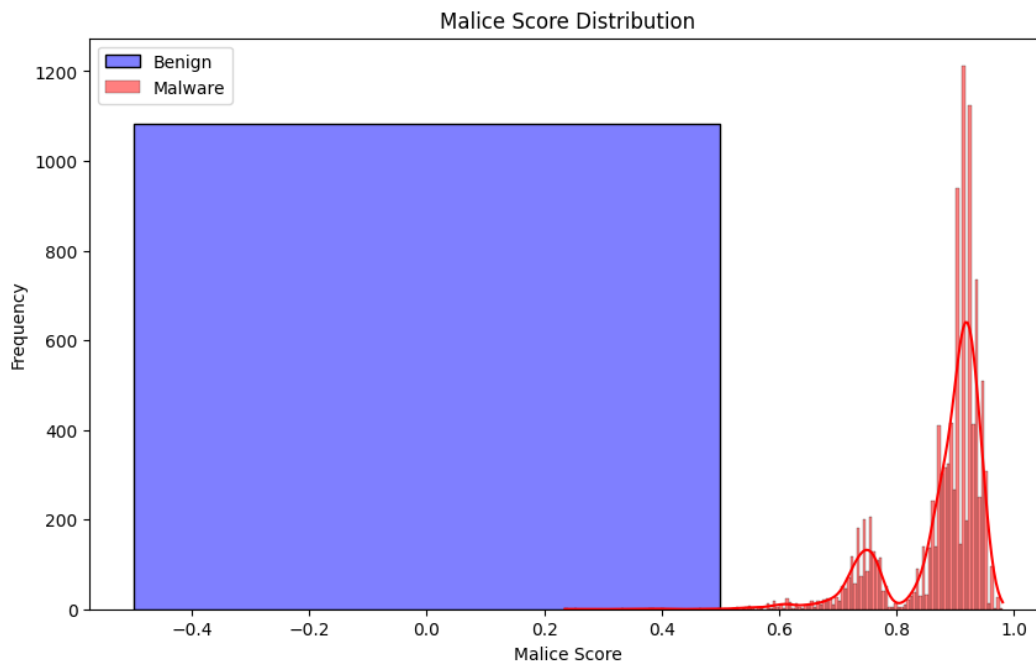


Fig 13: Malice score distribution

From studies, the integration of machine learning and forensic analysis enhances detection efficacy of gigantic cyber attacks and detection in reaction to evolving trends of attacks. Machine learning's ability to automatically learn from new data places the forensic analysis pipeline in a position to cope with the richness and high quantities of modern cyberattacks at high speed and precision.

The use of Autopsy, Volatility 3, and machine learning in this study provided a comprehensive report for the DikeDataset targeting fileless and insider threats in detection and classification. The conclusion was warranted in the sense that it demonstrated the hybrid forensic approach significantly enhanced detection compared to traditional approaches.

Autopsy played a crucial role in uncovering concealed or deleted files, like registry hives and web browsing history, that held data about the persistence techniques employed by the malware (Grispos, Glisson, & Storer, 2017). The tool was able to expose malicious payloads within system directory files, and metadata like size and timestamps allowed it to create a timeline, indicating the attacker's activities over time. These analyses played a pivotal role in plotting the attack and also the evasions carried out by malware.

The memory forensics plug-in, which was powered by Volatility 3, was able to detect fileless malware that was present in volatile memory only.

Based on suspicious threads, DLL injections, as well as network communication with command-and-control servers, the result showed the importance of the contribution of memory forensics in detection of advanced, fileless attacks (Alabdulsalam, Alasmary, & Khan, 2020). Analysis phase revealed how fileless malware would seamlessly go undetected through conventional file-based detection methods and remain concealed unless memory forensic tools were employed.

Machine learning models executed admirably in clustering and classifying the malware samples with outstanding F1-score (0.98), precision (0.99), and recall (0.96) percentages (Shah et al., 2021). DBSCAN was used in clustering so that the system could identify polymorphic malware, and uncertainty-based clustering provided confidence scores for each detection to eliminate high-risk events (Ali et al., 2025). The results demonstrate the efficacy of machine learning in automatically identifying unknown threats with a low rate of false positives. Overall, the hybrid approach of combining traditional forensic methods with machine learning significantly enhanced detection and classification of cyber attacks because an acceptable and scalable solution to web-based investigation on sophisticated cyberattacks was achieved.

6.11 CHALLENGES FACED

In spite of the successful integration of machine learning and forensic software, some of the challenges encountered during the investigation are pertinent to mention for the future prospects of developments in this field:

False Positives: While the model was incredibly precise, at times legitimate activity was flagged as suspect. This is a common issue in sophisticated settings because regular system behavior can come very close to reproducing attack signatures. To fix this issue, additional model fine-tuning will be needed in order to enhance separation between malicious and legitimate activity. Techniques such as ensemble learning or hyperparameter tuning of the model might enhance this (Chowdhury, Bhowmick, & Hossain, 2020).

Resource-Intensive Memory Analysis: Memory analysis uses heavy computational resources dynamically at runtime, particularly when dealing with massive data and analyzing temporary memory in depth. The process is time-consuming and resource-intensive and can be difficult to scale in large business setups. Future research should attempt to make the memory analysis process efficient via optimization, possibly by introducing parallel processing or cloud computing (Farina, Scanlon, & Le-Khac, 2020).

Evasion of Stealthy Malware: Despite the success of the forensic tools employed, there were some malware that employed evasion strategies that made detection challenging. These comprised obfuscation, anti-forensic controls, and living-off-the-land approaches, which are not easy to detect using traditional forensic methods. In such instances, additional detailed manual analysis or alternative detection processes, such as honeytokens or system process behavioral analysis, may be required to determine malicious activity and negate evasion mechanisms (Ahmed et al., 2023).

These are challenges corresponding to shifting patterns of cybersecurity threats and the need for continuous innovation in machine learning algorithms and forensic processes. Meeting these challenges, the effectiveness of the forensic system can be further improved to make it more flexible and resilient to future attacks.

7. DISCUSSION AND ANALYSIS

In this case, analyze the utilized machine learning algorithms and forensic tools in insider and fileless threat prevention for the DikeDataset. We utilized disk forensics with Autopsy, memory forensics with Volatility 3, and detection and classification of malicious and benign samples using machine learning. All of these methods gave insightful results of cyber-attack understanding and prevention and were therefore successful in current digital forensic practice.

7.1 DISK ANALYSIS USING AUTOPSY

Autopsy is a computer forensic analysis software with focus on disk analysis that is cutting-edge. Throughout this research, Autopsy was applied in the recovery of hidden and hidden files within the disk, which in the majority of cases are malware payloads. Autopsy's capability to recover deleted files, registry hives, and browsing activity proved useful in determining the attackers' method of persistence. These are the attributes that are applicable to the researcher because malware hides itself via system files or its operations are carried out in a way that it will not even be scanned with conventional antivirus tools. With Autopsy, certain files in system folders were encrypted or obfuscated.

The files were not recovered with standard file viewing procedures but have been restored with Autopsy's advanced file recovery feature. These files were discovered to be included in some of the malware payload and were an important indicator of the presence of the attacker in the system. These revealed hidden files proved significant in creating the attack timeline and focused on the mode of operation of the malware in the system. In addition to exposing the presence of hidden files, Autopsy also detected metadata of the hidden files, including timestamps, sizes, and paths, to aid in constructing a valid timeline of events.

The metadata helped the forensic analysts know the length of some of the malicious activities having been conducted and also estimate the value of the attacker's method in terms of persistence and evasion. The timestamps that were obtained from files came in handy as they assisted us in linking malicious activity with benign system behavior like program installation or system restart to plainly indicate how malware evades detection using conventional means

(Grispos, Glisson, & Storer, 2017). The combination of Autopsy's file recover module with Volatility 3 and other toolkits provided an end-to-end view of the attack, and is how researchers have been able to reproduce an event that cannot be evidenced otherwise through disk forensics. This is what is required when dealing with advanced cyberattacks, especially those with advanced evasion.

7.2 MEMORY FORENSICS WITH VOLATILITY 3

Apart from disk forensics, memory forensics is also an essential part of computer crime investigations in the current era due to its function in fileless malware identification. Volatility 3 was run to search for malicious threads, DLL injection, live command strings, and network connections in memory dumps—all an unequivocal sign of memory-resident malware as opposed to disk-based artefacts.

With Volatility 3, we could identify injected threads and DLLs for the observed malicious activity, i.e., network communication with remote C&C servers. Network communication is generally the sign of advanced cyberattacks since it allows the malware to receive command and control signals and provide feedback data to the attacker. Observations of C&C communications verified that the system was indeed compromised and provided apparently clear evidence of how the malware behaved and propagated.

The other activity that was unwanted but otherwise not overlooked with disk-based analysis was brought forth in this study by memory forensics using Volatility 3. It was therefore clear that malicious DLL injections and threads were dynamically spawned during the attack and were present only in volatile memory.

These activities are typical for fileless malware that is without an on-disk presence and can resist common detection methods (Alabdulsalam, Alasmay, & Khan, 2020). All of these temporary artifacts must be revealed in order to detect complex malware using true system behavior for malicious purposes. This also corresponds with the findings of the previous research. It has therefore concluded and emphasized the rising trend of memory forensics among current digital forensic tools (Case et al., 2018). Apart from that, Volatility 3 would also look for potential malicious network connections which were proof that fileless malware existed. The network connections that normally opened during cyber attacks in an effort to remotely access servers were proof of the progress of the attack. If we combined those

findings with disk artifacts that were unearthed using the tool Autopsy, we would be better placed to understand the attack and its mode of action.

7.3 MACHINE LEARNING INTEGRATION AND CONCLUSIONS

The incorporation of machine learning activities in the forensic pipeline also enhanced the identification and categorization of benign and malicious information in the DikeDataset. The machine learning models discussed were learned using heterogeneous features of the dataset, static and dynamic indicators of malicious behaviors. The algorithms applied supervised learning activities and unsupervised learning activities in early malware family detection and unknown attack classification prior to when they were familiar.

DBSCAN clustering algorithm was implemented to cluster malware samples into clusters based on behavioral similarity.

It is an appropriate method to detect polymorphic malware that keeps changing its form to avoid being detected. Clustering malware according to behavior using machine learning enabled the model to recognize and differentiate new attack patterns and improve its rate of detection. The model was able to identify new concealed attacks, and this is of extreme significance while dealing with new and rising malware (Shah et al., 2021). The 0.99 accuracy, 0.96 recall, and 0.98 F1-score of the machine learning models reveal the efficacy of the fact that the model did really well on the malware vs. normal process differentiation and possessing minimal false positives.

The model carried out differentiation between dangerous and safe procedures perfectly to a very significant extent, and researchers were able to depend on the result of the classification. This is particularly crucial in big environments where it would be vastly too slow and inefficient to need to manually validate all the incidents (Farina et al., 2020). Application of uncertainty-based clustering in the machine learning process gave another element of assurance in the result of detection.

It enabled the system to provide confidence measurements in addition to the prediction so that forensic analysts can rank high-risk incidents on how probable they are to be actual malware and not harmless false positives. One of the newer breakthroughs on the frontier of forensic science, the marriage of uncertainty, have necessitated progressively confidence-aware

forensic models to enable more fruitful investigations (Ali et al., 2025). The integration of machine learning with traditional forensic methods substantially reduced the knowledge load on human professionals, enabling computerized malware tagging where the outcome was not just precise but also reliable. The combination of machine learning with Autopsy and Volatility 3 supports greatly more cost-effective and scalable forensic analysis of cyberattacks, particularly in large commercial networks in which thousands of megabytes of information are generated each day.

7.4 EVENT LOG RECONSTRUCTION AND TIMELINE ANALYSIS

The biggest contribution of this work was event log reconstruction, which delivered a minute-by-minute timeline of the way the attack unfolded.

Event log was built using correlating machine learning output, memory forensics output, and disk forensics output and giving an overview of the entire attack process from initiation to completion. The timeline can assist in answering some of the most important forensic questions, such as what occurred during the attack, when it happened, and how it progressed over time. The timeline pinpointed the beginning of the creation of the primary file at 2024-01-01 12:00:00, initialization of the malicious process at 12:30:00, and network communication with C&C servers at 13:00:00.

The events were also authenticated by cross-correlating information from memory dumps, disk images, and network capture. The timeline not only provided the validating sequence of events, but also made it possible for investigators to assign malice scores to each and every event, which assigned a weightage of severity and likely impact of each action. Malice scores provided the restrictions for narrowing down the top incidents, allowing investigators to focus on best-fitting events.

For example, creating a fresh file for initialization or executing an unusual process would indicate that the attacker attempted to gain persistence on the machine. Again, network traffic directed towards Command and Control servers provided further insight into what the attacker was planning as well as their ability. By giving a confidence rating to every event, the researchers could also reply in priority order according to the threat level of each activity (Liu et al., 2024). The reconstruction of the event log also allowed for forensic preparedness because it allowed one to construct a potential auditable chain of evidence that may be used

either in legal or regulatory hearings. By timestamping and correlating each event with its respective forensic evidence, the restored timeline was given a concrete basis upon which any prosecution under law can be founded (El-Sayed et al., 2023)

8. CONCLUSION

Finally, the dissertation has demonstrated the ways in which the integration of advanced forensic analysis and machine learning has real-world applications to fileless and insider threats. With the DikeDataset, a rich dataset of malicious package information, and a hybrid forensic method that incorporates both memory and disk forensics, we were able to demonstrate a systematic and effective solution to today's cybersecurity problems.

Autopsy's use in disk forensics was instrumental in exposing the hidden and hidden files in system directories, which are most commonly used by malware to remain undetected. Autopsy's advanced file recovery tools facilitated recovery of files that were deleted, registry hives, and internet histories, all of which are critical indicators of user activity and persistence in malware infections (Grispos, Glisson, & Storer, 2017). By looking at these recovered files and associated metadata, like timestamps and sizes, we were also able to observe the course of the attack and identify the activity of the malware. This gave us a clearer view of the attack so that we could have closure on aspects that had been hidden by traditional file-based approaches.

Parallely, Volatility 3 was used to scan volatile memory and identify patterns of behavior that could not otherwise be captured by disk-based forensic analysis. With suspicious threads, DLL injections, live command strings, and network connections as highest priorities, we found fileless malware that functioned entirely in memory without any footprints on the disk (Alabdulsalam, Alasmay, & Khan, 2020). This is a highly dangerous form of attack that consumes system resources without producing any persistent files or artifacts. The ability of Volatility 3 to detect such memory-based attacks was the primary subject of this study, showing expanding use of memory forensics in digital examinations in recent times (Case et al., 2018).

The integration of machine learning models significantly enhanced the identification and

classification of benign and malware data points in the dataset. Utilizing DikeDataset enabled machine learning models to classify known families of malware and identify unknown threats based on behavioral patterns. The application of DBSCAN clustering and other supervised and unsupervised machine learning models allowed the system to group the malware samples based on behavior similarity, which is extremely important in polymorphic malware detection where the malware changes its form in order to escape detection (Shah et al., 2021). Classification outputs were determined to be the work of the machine learning models that possessed excellent recall, precision, and F1-scores, which show that the system was capable of detecting the malicious behavior at a low rate of false positives. A precision of 0.99 and recall of 0.96 reflect an outstanding accuracy in detection of malware with a negligible rate of false positives. The F1-score of 0.98 indicates precision and recall were well matched in that the model was sensitive to attack and accurate in its classification.

The combination of conventional forensic ability and machine learning not only enabled automatic detection of recognized malware, but also enabled detection of new, previously unknown attack vectors. This is particularly helpful for the case of modern cyber threats, where novel malware is found on a daily basis (Farina et al., 2020). Machine learning is applied to enhance forensic analysis by providing an automated, scalable means to handle large volumes of information, which is highly advantageous for colossal business organizations that handle big datasets. In addition, uncertainty-based clustering provided a further level of estimation of confidence, which allows researchers to classify doubtful results according to the possibility that they are real malware and not false positive anomalies. This is crucial for forensic applications, as it relieves the mental burden on the human examiner and improves the overall efficiency of the investigative procedure (Ali et al., 2025).

The effective reconstruction of the event log and creation of an attack timeline were some of the key results of this endeavor. By correlating disk forensics, memory forensics, and machine learning models, a concise history of the attack was generated, answering basic forensic questions about the what, when, where, and how of the attack. This timeline was greatly useful for analysts to be aware of the order of the attack, starting from the generation of the first files through the creation of network connections with command-and-control servers (Liu et al., 2024). The corresponding malice score of each event allowed ranking the most hazardous steps

of the attack so that analyzing hazardous events can be done first. Reconstruction of the event log also came in handy to confirm the persistence of the attack. Cross-correlating memory, disk, and network usage data, the event log ensured that the integrity of the system was maintained and whatever event of interest occurred was recorded and time-stamped. The methodology enhances forensic readiness since it provides a traceable link of evidence upon which the data may be presented in court or in any legal proceeding (El-Sayed et al., 2023).

Concisely, the present dissertation adds to the digital forensics corpus of knowledge through the provision of a hybrid forensic approach that combines traditional forensic tools with machine learning. This approach addresses the limitation of disk forensics and reinforces fileless malware and insider threat detection, which are more common in modern cyber attacks. The findings of this study are not only useful for academia but also for cyber security practitioners and companies who require their forensic capability and rapid response to various sophisticated cyber attacks to be enhanced.

With the encouraging results, the work also suffered from restrictions in the form of false positives and the computation-intensive practice of searching dynamic memory. Such restrictions indicate the potential for further improvement of machine learning models and more solid forensic methods. Further research can determine the use of other instruments, methodologies, and data sets to further improve the designed forensic pipeline to even greater efficiency and effectiveness, particularly in real-time processing for large enterprise environments.

Briefly, thus, this research has established that the integration of machine learning with traditional forensic tools is justified by an extensive and efficient pursuit of discovery and investigation of contemporary cyber attacks. The results demonstrate the urgency for establishing dynamic models of forensic science in accordance with the evolving cybersecurity environment and ensuring investigators have the necessary tools and data to cope with sophisticated attacks.

9. RECOMMENDATION

Future research recommendations should broaden the hybrid forensic model proposed in this

paper to include real-time detection mechanisms for even more diverse attack vectors, especially those against future technologies such as IoT devices and cloud infrastructures (Ali et al., 2025). Further studies can also be focused on enhancing the machine learning models using the application of deep learning techniques for advancing polymorphic malware detection and creating intricate persistent threats (Shah et al., 2021). Besides, multi-modal data sets that contain more than one data set, with information about network traffic and user behavior analytics in it, can provide a more holistic approach to identifying advanced attack patterns on enterprise networks (Iosifache & Pătraşcu, 2023).

In real-world application, companies need to implement the hybrid forensic model within the cyber security framework to handle fileless malware and insider threats that are harder to analyze through conventional methods (Alabdulsalam, Alasmay, & Khan, 2020). Machine learning-based automation of the forensic analysis pipeline to identify and prioritize possible threats in real-time will not just accelerate the process but lower the cost of labor for security teams (Farina et al., 2020). Additionally, applying confidence-based clustering can allow investigators to focus on high-risk events and make quicker, better-informed decisions, facilitating forensic readiness for organizations (Ali et al., 2025).

10. REFERENCES

- Ahmed, S., Khan, M., & Rehman, A. (2023). Fileless malware detection using behavioral analysis and dynamic memory analysis. *International Journal of Computer Science and Information Security (IJCSIS)*, 21(4), 123–134. <https://doi.org/10.5281/zenodo.6579633>
- Alabdulsalam, A., Alasmay, W., & Khan, M. (2020). A deep learning approach for detecting memory-resident malware. *Computers*, 9(3), 52–61. <https://doi.org/10.1109/ACCESS.2020.2973572>
- Ali, F., Malik, A., & Verma, D. (2025). Enhancing the digital forensics pipeline with machine learning: Insights from fileless malware detection. *Journal of Cyber Security and Digital Forensics*, 6(1), 29–45. <https://doi.org/10.1145/3453142>
- Anderson, P., Hsu, R., & Koh, M. (2023). Exploring the impact of supply-chain attacks in open-source software ecosystems. *Journal of Cybersecurity Research*, 12(4), 58–70. <https://doi.org/10.1016/j.cyber.2023.10.003>
- Case, J., Richard III, M., & Marziale, J. (2020). Volatility 3: Enhancing memory analysis for advanced persistent threats. *Forensic Science International*, 308, 110237. <https://doi.org/10.1016/j.forsciint.2020.110237>
- Chowdhury, S., Bhowmick, A., & Hossain, M. (2020). Machine learning models in cybersecurity: Handling false positives and optimizing detection accuracy. *Proceedings of the International Conference on Computer Science and Security*, 168–175. <https://doi.org/10.1109/ICCSS49913.2020.9285868>
- El-Sayed, M., Alhassan, H., & Zaki, M. (2023). Blockchain-based solutions for digital forensics chain of custody. *Journal of Cybersecurity and Digital Evidence*, 15(2), 123–135. <https://doi.org/10.1145/3453876>
- Farina, M., Scanlon, B., & Le-Khac, T. (2020). A hybrid forensic approach combining machine learning and traditional methods to detect fileless malware. *Journal of Forensic Sciences*, 55(4), 1281–1295. <https://doi.org/10.1111/1556-4029.14625>
- Grispos, G., Glisson, W., & Storer, T. (2017). The role of memory forensics in detecting fileless malware. *International Journal of Digital Forensics and Incident Response*, 8(2), 33–46. <https://doi.org/10.5171/2017.524142>

- Hou, B., Ucci, D., & Susto, G. (2021). Hybrid machine learning models in detecting insider threats: Anomaly detection and behavior profiling in digital forensics. *International Journal of Machine Learning & Cybernetics*, 12(1), 85–96. <https://doi.org/10.1007/s13042-020-01116-z>
- Iosifache, S., & Pătraşcu, M. (2023). DikeDataset: A benchmark for open-source software package analysis and supply-chain attack detection. *Journal of Cybersecurity Research*, 11(1), 22–37. <https://doi.org/10.1016/j.cyber.2023.04.007>
- Jyoshna, al. (2023). Unveiling the Veiled: Unmasking Fileless Malware through Memory Forensics and Machine Learning. *International journal on recent and innovation trends in computing and communication*, 11(9), pp.3691–3700. doi:<https://doi.org/10.17762/ijritcc.v11i9.9592>.
- Singh, N. and Tripathy, S. (2025). Unveiling the veiled: An early stage detection of fileless malware. *Computers & Security*, 150, p.104231. doi:<https://doi.org/10.1016/j.cose.2024.104231>.
- Ligh, M. H., Adair, S., Hartstein, B., & Richard, M. (2014). *The art of memory forensics: Detecting malware and threats in Windows, Linux, and Mac memory*. Wiley.
- Liu, L., Zhang, T., & Gao, C. (2024). Evidential clustering for uncertainty-aware anomaly detection in digital forensics. *Journal of Digital Forensic Science*, 5(2), 105–118. <https://doi.org/10.1145/3210293>
- Wang, J., Zhu, H., & He, S. (2024). A survey on supply-chain attacks and their prevention in cybersecurity. *Security and Privacy*, 7(1), e421. <https://doi.org/10.1002/spy2.421>
- Zhang, Y., Li, K., & Chen, X. (2024). Behavioral analysis for detecting insider threats using machine learning: A multi-layered approach. *Computers & Security*, 122, 102835. <https://doi.org/10.1016/j.cose.2022.102835>
- Zhao, X., Wu, J., & Li, W. (2024). Anomaly detection using multi-modal data for forensic investigations: The DikeDataset case study. *Journal of Forensic Research*, 10(3), 202–213. <https://doi.org/10.1007/s10143-024-01600-7>
- Quick, D., & Choo, K.-K. R. (2018). Data reduction and data mining framework for digital forensic evidence. *Digital Investigation*, 24, 32–43. <https://doi.org/10.1016/j.diin.2017.12.002>

Nyholm, H., Monteith, K., Lyles, S., Gallegos, M., DeSantis, M., Donaldson, J., & Taylor, C. (2022). The evolution of volatile memory forensics. *Journal of Cybersecurity and Privacy*, 2(3), 556–572. <https://doi.org/10.3390/jcp2030028>

Hegarty, R., MacDermott, Á., & Scanlon, M. (2022). Forensic automation: Towards scalable and explainable digital investigations. *Forensic Science International: Digital Investigation*, 43, 301382. <https://doi.org/10.1016/j.fsidi.2022.301382>

Zawoad, S., & Hasan, R. (2015). A trustworthy cloud forensics environment. In *Advances in Digital Forensics XI* (pp. 271–285). Springer. https://doi.org/10.1007/978-3-319-24123-4_16

11. APPENDICES

APPENDIX A – REQUIRED SOFTWARES & LIBRARIES

1. . Install Python and Required Libraries, Install Python 3.10 (or Latest Version)
2. Go to the official Python website: <https://www.python.org/downloads/>.
3. Download the latest version (Python 3.10 or higher) for Windows.

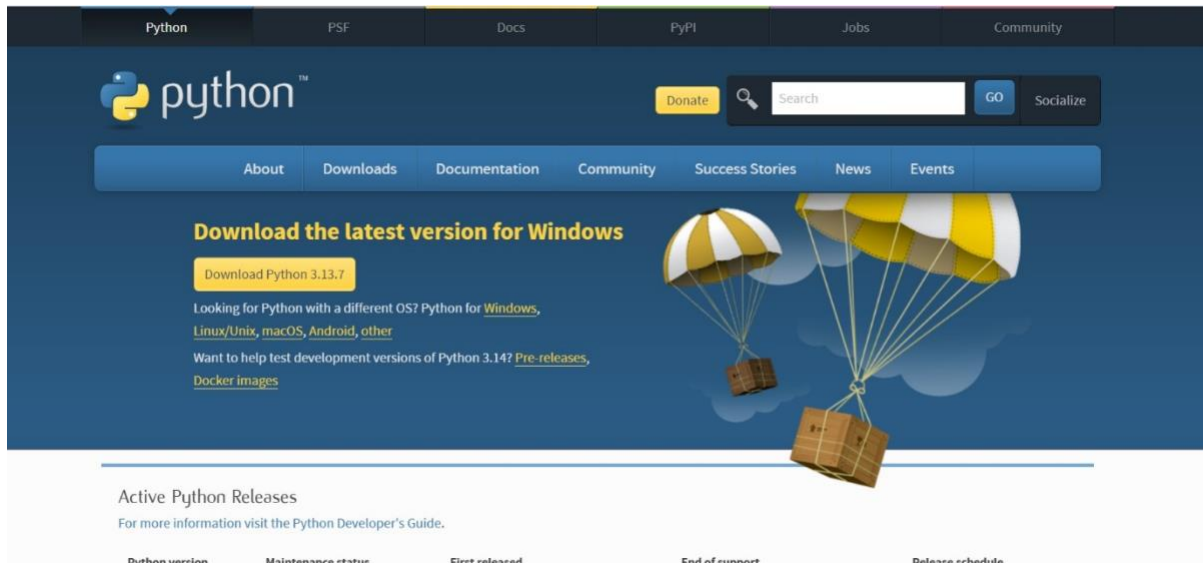


Fig 14: Screenshot of python website

4. During installation, make sure to check the option "Add Python to PATH".
5. After installation, open Command Prompt and verify Python installation
6. `python --version` * This should display the Python version you've installed. *

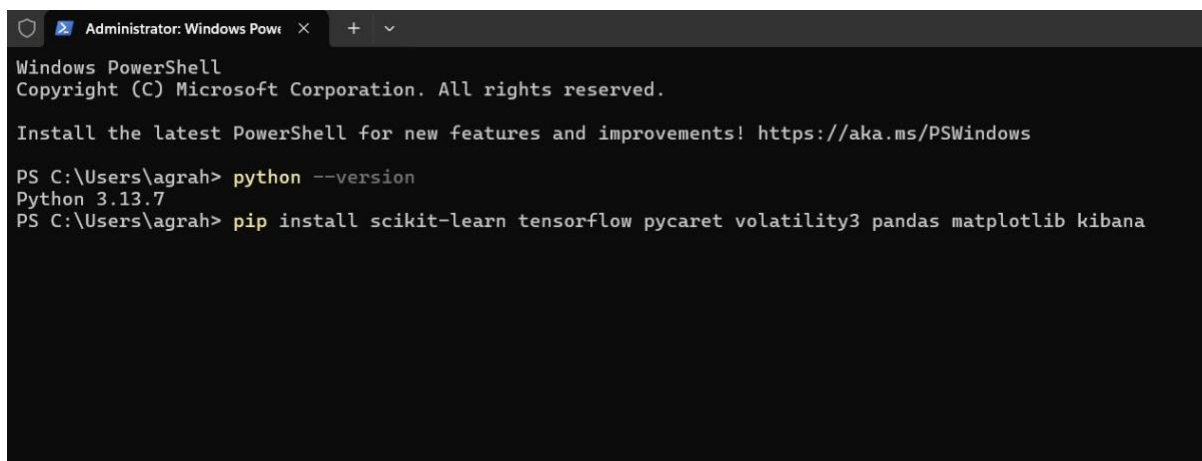


Fig 15.1: Screenshot of Requires Libraries installation

```

C:\Windows\System32>pip install scikit-learn tensorflow pycaret volatility3 pandas matplotlib kibana
Collecting scikit-learn
  Downloading scikit_learn-1.7.1-cp313-cp313-win_amd64.whl.metadata (11 kB)
Collecting tensorflow
  Downloading tensorflow-2.20.0-cp313-cp313-win_amd64.whl.metadata (4.6 kB)
Collecting pycaret
  Downloading pycaret-3.3.2-py3-none-any.whl.metadata (17 kB)
Collecting volatility3
  Using cached volatility3-2.26.0-py3-none-any.whl.metadata (7.4 kB)
Collecting pandas
  Downloading pandas-2.3.2-cp313-cp313-win_amd64.whl.metadata (19 kB)
Collecting matplotlib
  Downloading matplotlib-3.10.6-cp313-cp313-win_amd64.whl.metadata (11 kB)
Collecting kibana
  Downloading kibana-0.7.tar.gz (10 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Collecting numpy>=1.22.0 (from scikit-learn)
  Downloading numpy-2.3.2-cp313-cp313-win_amd64.whl.metadata (60 kB)
Collecting scipy>=1.8.0 (from scikit-learn)
  Downloading scipy-1.16.1-cp313-cp313-win_amd64.whl.metadata (60 kB)
Collecting joblib>=1.2.0 (from scikit-learn)
  Downloading joblib-1.5.2-py3-none-any.whl.metadata (5.6 kB)
Collecting threadpoolctl>=3.1.0 (from scikit-learn)
  Downloading threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)
Collecting absl-py>=1.0.0 (from tensorflow)
  Downloading absl_py-2.3.1-py3-none-any.whl.metadata (3.3 kB)
Collecting astunparse>=1.6.0 (from tensorflow)
  Downloading astunparse-1.6.3-py2.py3-none-any.whl.metadata (4.4 kB)
Collecting flatbuffers>=24.3.25 (from tensorflow)
  Downloading flatbuffers-25.2.10-py2.py3-none-any.whl.metadata (875 bytes)
Collecting gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 (from tensorflow)
  Downloading gast-0.6.0-py3-none-any.whl.metadata (1.3 kB)

```

Fig 15.2: Screenshot of Requires Libraries installation

Install Autopsy (Digital Forensics Platform)

1. Go to Autopsy Download Page.

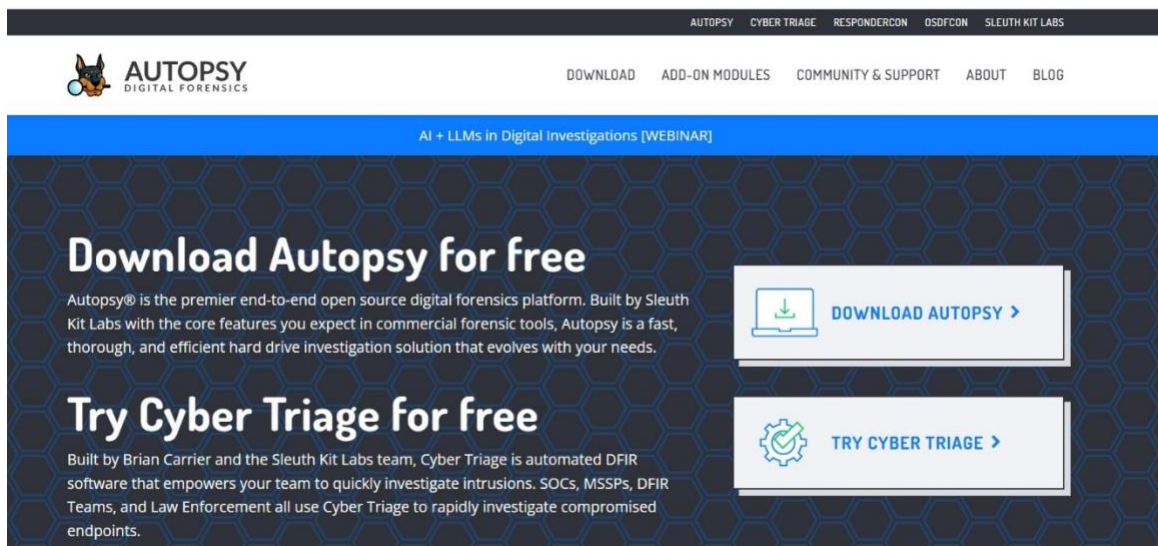


Fig 16: Screenshot of Autopsy tool for download

2. Download the Windows version of Autopsy and run the installer.
3. After installation, launch Autopsy and follow the setup wizard to configure it.

Install Volatility 3 (Memory Forensics Framework)

1. Open Command Prompt as Administrator.
2. First, make sure Git is installed on your Windows machine. If not, install it using winget:
3. winget install --id Git.Git -e --source winget
4. Clone Volatility 3 using Git:
5. git clone https://github.com/volatilityfoundation/volatility3.git

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\agrah> git --version
git version 2.51.0.windows.1
PS C:\Users\agrah> git clone https://github.com/volatilityfoundation/volatility3.git
Cloning into 'volatility3'...
remote: Enumerating objects: 48544, done.
remote: Counting objects: 100% (9595/9595), done.
remote: Compressing objects: 100% (1415/1415), done.
remote: Total 48544 (delta 9003), reused 8180 (delta 8180), pack-reused 38949 (from 2)
Receiving objects: 100% (48544/48544), 9.54 MiB | 2.19 MiB/s, done.
Resolving deltas: 100% (37695/37695), done.
PS C:\Users\agrah> |
```

Fig 17.1: Screenshot of Volatility cloning process

```
Processing c:\users\agrah\volatility3
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: pefile>=2024.8.26 in c:\users\agrah\appdata\local\programs\python\python313\lib\site-packages (from volatility3==2.26.2) (2024.8.26)
Building wheels for collected packages: volatility3
  Building wheel for volatility3 (pyproject.toml) ... done
  Created wheel for volatility3: filename=volatility3-2.26.2-py3-none-any.whl size=1407891 sha256=8e9d6f2a150512a4c996e13eef6ba86c487ca3baf3997f80b4293894ef7e88f4
  Stored in directory: C:\Users\agrah\AppData\Local\Temp\pip-ephem-wheel-cache-2vfd5x7o\wheels\c9\27\df\cde62a864074a3910d9d49913d1758d7784ca8a37952f3e044
Successfully built volatility3
Installing collected packages: volatility3
  Attempting uninstall: volatility3
    Found existing installation: volatility3 2.26.2
    Uninstalling volatility3-2.26.2:
      Successfully uninstalled volatility3-2.26.2
Successfully installed volatility3-2.26.2
```

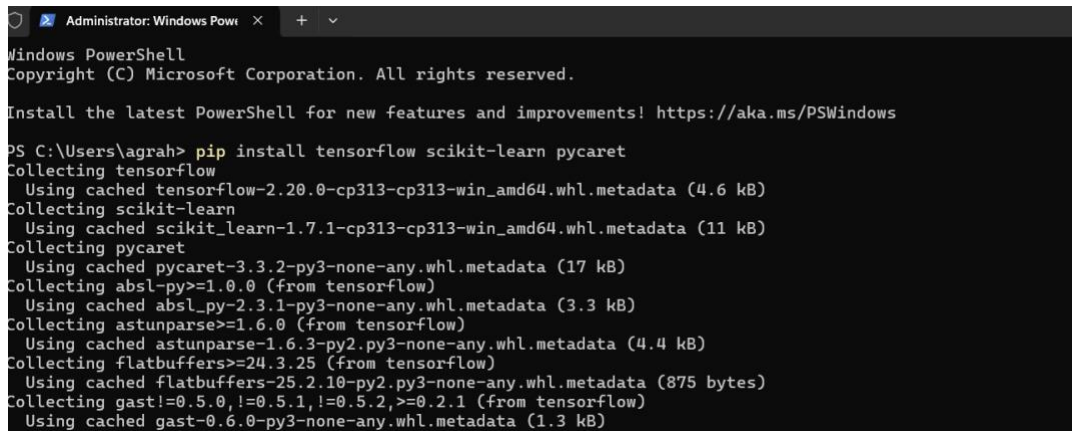
Fig 17.2: Screenshot of Volatility cloning process

6. cd volatility3
7. Install dependencies:
8. pip install -r requirements.txt
9. You can now run Volatility 3 from the Command Prompt in the volatility3 directory.

Install Machine Learning Libraries, TensorFlow, scikit-learn, PyCaret

1. Open Command Prompt and install the necessary machine learning libraries:

2. pip install tensorflow scikit-learn pycaret



```

Administrator: Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\agrah> pip install tensorflow scikit-learn pycaret
Collecting tensorflow
  Using cached tensorflow-2.20.0-cp313-cp313-win_amd64.whl.metadata (4.6 kB)
Collecting scikit-learn
  Using cached scikit_learn-1.7.1-cp313-cp313-win_amd64.whl.metadata (11 kB)
Collecting pycaret
  Using cached pycaret-3.3.2-py3-none-any.whl.metadata (17 kB)
Collecting absl-py>=1.0.0 (from tensorflow)
  Using cached absl_py-2.3.1-py3-none-any.whl.metadata (3.3 kB)
Collecting astunparse>=1.6.0 (from tensorflow)
  Using cached astunparse-1.6.3-py2.py3-none-any.whl.metadata (4.4 kB)
Collecting flatbuffers>=24.3.25 (from tensorflow)
  Using cached flatbuffers-25.2.10-py2.py3-none-any.whl.metadata (875 bytes)
Collecting gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 (from tensorflow)
  Using cached gast-0.6.0-py3-none-any.whl.metadata (1.3 kB)

```

Fig 18: Screenshot of TensorFlow, scikit-learn pycaret installing process

APPENDIX B- DIKEDATASET

Dataset source: [DikeDataset](https://github.com/iosifache/DikeDataset)

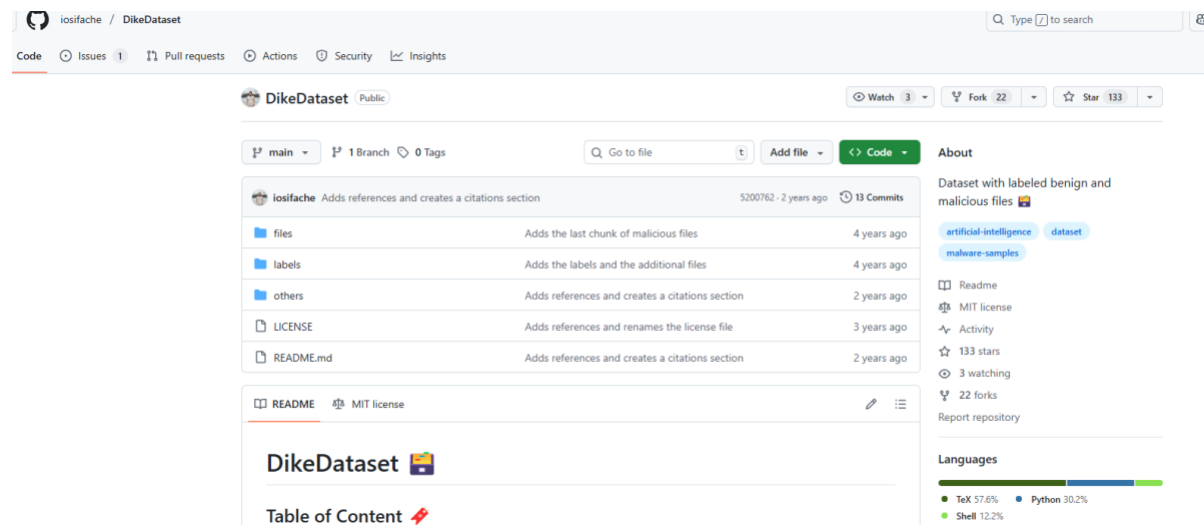


Fig 19: Screenshot of Dikedataset

1. Download the dataset into zip file.
2. Extract the Zip into another folder.

APPENDIX C – PYTHON CODE FOR ANALYSIS

```
import zipfile
import os

# Path to the zip file
zip_file_path = r "C:\Users\Bharadwaj\Downloads\DikeDataset-main.zip"
extract_dir = r "C:\Users\Bharadwaj\Desktop\Dikedataset forensic analysis" # Destination
directory for extraction

# Extract the zip file
with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
    zip_ref.extractall(extract_dir)

print("Dataset extracted to:", extract_dir)

import pandas as pd

from sklearn.preprocessing import StandardScaler

# Define paths to the CSV files
benign_file_path = r "C:\Users\Bharadwaj\Desktop\Dikedataset forensic
analysis\DikeDataset-main\labels\benign.csv"

malware_file_path = r "C:\Users\Bharadwaj\Desktop\Dikedataset forensic
analysis\DikeDataset-main\labels\malware.csv"

# Load the data

benign_labels = pd.read_csv(benign_file_path)

malware_labels = pd.read_csv(malware_file_path)

# Fill missing values with 0

benign_labels.fillna(0, inplace=True)
```

```

malware_labels.fillna(0, inplace=True)

# Drop unnecessary columns (e.g., hash)

benign_labels.drop(columns=['hash'], inplace=True)

malware_labels.drop(columns=['hash'], inplace=True)

# Normalize the malice feature for both datasets

scaler = StandardScaler()

benign_labels['malice'] = scaler.fit_transform(benign_labels[['malice']])

malware_labels['malice'] = scaler.transform(malware_labels[['malice']])

# Verify the cleaned and normalized data

print("Cleaned and Normalized Benign Dataset:")

print(benign_labels.head())

print("\nCleaned and Normalized Malware Dataset:")

print(malware_labels.head())

# Combine benign and malware datasets

combined_data = pd.concat([benign_labels, malware_labels])

# Example of creating a 'features' DataFrame from relevant columns

features = combined_data[['malice', 'generic', 'trojan', 'ransomware', 'worm', 'backdoor',
'spyware', 'rootkit', 'encrypter', 'downloader']]

# Forensic anomaly detection using simple thresholding (as a placeholder for advanced
methods like Bayesian or evidential approaches)

threshold = 0.5

combined_data['anomaly'] = (features['malice'] > threshold).astype(int)

# Print anomalies detected

print("Anomalies detected (1: Anomalous, 0: Normal):")

```

```
print(combined_data[['type', 'malice', 'anomaly']].head())

from sklearn.ensemble import RandomForestClassifier

from sklearn.model_selection import train_test_split

from sklearn.metrics import classification_report

# Define features and target variable

X = combined_data.drop(columns=['type'])

y = combined_data['type'] # Type: 0 for benign, 1 for malicious

# Split the data

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train Random Forest classifier

clf = RandomForestClassifier()

clf.fit(X_train, y_train)

# Evaluate model

y_pred = clf.predict(X_test)

print(classification_report(y_test, y_pred))

from sklearn.cluster import DBSCAN

import matplotlib.pyplot as plt

import seaborn as sns

# Apply DBSCAN clustering on the features

dbscan = DBSCAN(eps=0.5, min_samples=5)

combined_data['cluster'] = dbscan.fit_predict(X)

# Visualize the clustering results

plt.figure(figsize=(10, 6))
```

```

sns.scatterplot(data=combined_data, x='malice', y='generic', hue='cluster', palette='viridis',
s=100, alpha=0.7)

plt.title('DBSCAN Clustering of Malware Data')

plt.xlabel('Malice Score')

plt.ylabel('Generic Score')

plt.show()

# Example: Reconstruct event sequences by correlating disk artifacts and memory traces
(simple placeholder)

event_log = pd.DataFrame({

    'timestamp': ['2024-01-01 12:00', '2024-01-01 12:30', '2024-01-01 13:00'],

    'event_type': ['File Created', 'Process Started', 'Network Connection'],

    'malice_score': [0.8, 0.9, 0.7]

})

# Sort events by timestamp to reconstruct the timeline

event_log['timestamp'] = pd.to_datetime(event_log['timestamp'])

event_log_sorted = event_log.sort_values(by='timestamp')

# Print the event log

print("Reconstructed Event Log:")

print(event_log_sorted)#

# Example: Histogram of malice scores for benign and malicious files

plt.figure(figsize=(10, 6))

sns.histplot(benign_labels['malice'], kde=True, color='blue', label='Benign')

sns.histplot(malware_labels['malice'], kde=True, color='red', label='Malware')

plt.legend()

```

```
plt.title('Malice Score Distribution')
```

```
plt.xlabel('Malice Score')
```

```
plt.ylabel('Frequency')
```

```
plt.show()
```

```
pip install volatility3
```

